



**Eötvös Loránd Tudományegyetem**

**Informatikai Kar**

**Informatikatudományi Intézet**

**Komputeralgebra Tanszék**

# Vendégház weboldala Laravel keretrendszerben

**Szerző:**

Orbán László

Programtervező informatikus BSc.

**Témavezető:**

Magyar Péter

Egyetemi Tanársegéd,

Programtervező informatikus MSc

**Szombathely, 2026**

**EÖTVÖS LORÁND TUDOMÁNYEGYETEM**  
INFORMATIKAI KAR

**SZAKDOLGOZAT TÉMABEJELENTŐ**

**Hallgató adatai:**

Név: Orbán László

Neptun kód: E8XPX0

**Képzési adatok:**

Szak: programtervező informatikus, alapképzés (BA/BSc/BProf)

Tagozat: Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Magyar Péter

munkahelyének neve: Eötvös Loránd Tudományegyetem, Informatikai Kar, Komputeralgebra Tanszék

munkahelyének címe: 9700 Szombathely, Károlyi Gáspár tér 4.

beosztás és iskolai végzettsége: Egyetemi Tanársegéd, Programtervező informatikus MSc

**A szakdolgozat címe:** Vendégház weboldala Laravel keretrendszerben

**A szakdolgozat témája:**

*(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben szakdolgozat témájának leírását )*

A szakdolgozat célja egy családi vendégház számára készülő, korszerű, reszponzív és biztonságos weboldal fejlesztése Laravel keretrendszerben. A weboldal célja, hogy egy vendégház számára professzionális online jelenlétet biztosítson, megkönnyítse a vendégek tájékozódását, valamint egyszerűsítse a foglalások és érdeklődések kezelését.

A rendszer MySQL adatbázist használ az adatok tárolására, az alkalmazás fejlesztése során pedig a Laravel beépített ORM (Eloquent) eszközeit használja az adatok biztonságos és hatékony kezeléséhez. A felhasználói felület kialakításához Bootstrap CSS keretrendszert alkalmaz, amely támogatja a reszponzív, mobilbarát megjelenést, és gyors fejlesztést tesz lehetővé.

A weboldal fő funkciói:

- a vendégház részletes bemutatása (leírás, képek, elérhetőségek),
- szobák és árak listázása, részletes információkkal,
- online foglalási vagy érdeklődési űrlap, amely e-mail értesítést küld a tulajdonosnak,
- kapcsolatfelvételi űrlap validációval és biztonságos adatkezeléssel,
- adminisztrációs felület, ahol a vendégház adatai (szobák, képek, árak, foglalások) kezelhetők,
- jelszóval védett bejelentkezési rendszer az admin funkciókhoz.

Budapest, 2025. 10. 15.

# Tartalomjegyzék

|        |                                                                             |    |
|--------|-----------------------------------------------------------------------------|----|
| 1.     | Bevezetés .....                                                             | 1  |
| 2.     | Felhasználói dokumentáció.....                                              | 2  |
| 2.1.   | Rendszerkövetelmények és hozzáférés: .....                                  | 2  |
| 2.2.   | A publikus felület (Vendég nézet): .....                                    | 2  |
| 2.2.1. | Navigáció és információs oldalak bemutatása .....                           | 2  |
| 2.2.2. | A foglalás menete .....                                                     | 4  |
| 2.3.   | Az Adminisztrációs felület bemutatása .....                                 | 7  |
| 2.3.1. | Hitelesítés és biztonság .....                                              | 7  |
| 2.3.2. | Vezérlőpult és Foglalások kezelése .....                                    | 7  |
| 2.3.3. | Globális beállítások és árak dinamikus kezelése .....                       | 9  |
| 2.3.4. | Tartalomkezelés (Galéria és Szobák) .....                                   | 10 |
| 3.     | Fejlesztői dokumentáció .....                                               | 12 |
| 3.1.   | Felhasznált technológiák és fejlesztői környezet .....                      | 12 |
| 3.1.1. | Fejlesztői környezet és verziókezelés .....                                 | 12 |
| 3.1.2. | Szerveroldali technológiák .....                                            | 12 |
| 3.1.3. | Adatbázis-kezelés .....                                                     | 13 |
| 3.1.4. | Kliensoldali technológiák .....                                             | 13 |
| 3.1.5. | Felhasználói felület és prototipizálás .....                                | 14 |
| 3.2.   | Rendszerarchitektúra és Üzleti logika .....                                 | 14 |
| 3.2.1. | Az MVC architekturális minta a gyakorlatban .....                           | 14 |
| 3.2.2. | Osztálystruktúra és UML modellezés .....                                    | 15 |
| 3.2.3. | Az üzleti logika .....                                                      | 18 |
| 3.2.4. | Dinamikus árkalkuláció .....                                                | 18 |
| 3.2.5. | Útválasztás ( <i>Routing</i> ) és Köztesrétegek ( <i>Middleware</i> ) ..... | 21 |
| 3.3.   | Adatbázis-tervezés és Adatkezelés .....                                     | 22 |

|        |                                                                          |    |
|--------|--------------------------------------------------------------------------|----|
| 3.3.1. | Relációs adatmodell és ER-diagram .....                                  | 22 |
| 3.3.2. | Adatbázis-migrációk és az Eloquent ORM a gyakorlatban.....               | 23 |
| 3.3.3. | Adatintegritás, GDPR és a törlési stratégiák .....                       | 24 |
| 3.4.   | Biztonsági Architektúra .....                                            | 25 |
| 3.4.1. | Adatbeviteli sebezhetőségek és jogosultságkezelés.....                   | 25 |
| 3.4.2. | Többrétegű validáció és a Zéró Bizalom ( <i>Zero Trust</i> ) modell..... | 26 |
| 3.4.3. | Biztonságos HTTP kommunikáció és tranzakcióvédelem .....                 | 27 |
| 3.5.   | Teljesítményoptimalizálás és Felhasználói Élmény (UX/UI).....            | 28 |
| 3.5.1. | Backend teljesítmény és erőforrás-kezelés.....                           | 28 |
| 3.5.2. | Core Web Vitals és Frontend ergonómia .....                              | 28 |
| 3.5.3. | Kognitív terhelés minimalizálása és Zökkenőmentes UX (Seamless UX) ..... | 29 |
| 4.     | A rendszer tesztelése .....                                              | 31 |
| 4.1.   | Üzleti logika és funkcionális tesztek.....                               | 31 |
| 4.2.   | Biztonsági tesztelés és a Zéró Bizalom .....                             | 32 |
| 5.     | Összefoglaló .....                                                       | 34 |
| 5.1.   | Konklúzió .....                                                          | 34 |
| 5.2.   | Továbbfejlesztési lehetőségek .....                                      | 35 |
| 6.     | Irodalomjegyzék .....                                                    | 37 |

## 1. Bevezetés

A szakdolgozat motivációja egy modern, reszponzív weboldal létrehozása a Mura Vendégház számára, amely azon túl, hogy bemutatja a szállást, automatizálja a foglalást, és lehetővé teszi az üzemeltetőnek a honlap adatainak programozói beavatkozás nélküli szerkesztését.

A főbb célkitűzések között szerepelt többek között a figyelemfelhívó, könnyen kezelhető, átlátható, modern publikus felület a vendégeknek, a GDPR-kompatibilis adatkezelés, a dinamikus áralkuláció, és egy adminisztrációs felület. Utóbbin keresztül a vendégház tulajdonosának módjában kell állnia megváltoztatni az árakat és a globális beállításokat, módosítani a főbb lakóegységek leírását és képeit, szerkeszteni a galéria képeit, illetve kezelni a bejövő foglalásokat.

A megvalósítás alapját a PHP [1] nevű, széles körben használt szerveroldali programozási nyelv adja. Ez kezeli a felhasználói kéréseket, és a háttérben összeállítja a tartalmat a böngészőnek. A gyorsabb, átláthatóbb fejlesztés, és a kevesebb hibalehetőség érdekében egy modern és népszerű PHP keretrendszert, a Laravelt [2] alkalmaztam. Ez adott egy előre felépített, biztonságos alapot és eszközkészletet a munka megkönnyítéséhez. A reszponzív, mobil- és tabletbarát weboldal eléréshez a Bootstrap [3] nevű frontend (felhasználói felület) keretrendszert használtam. Az előre elkészített dizájnelemekkel jelentős mértékben gyorsította a fejlesztés folyamatát. A projekthez szükségem volt egy megbízható relációs adatbázis-kezelő rendszerre, amit a MySQL [4] biztosított. Ez felel a weboldal adatainak biztonságos, táblázatos és logikus formában történő tárolásáért, valamint a gyors adatkikérésért.

## 2. Felhasználói dokumentáció

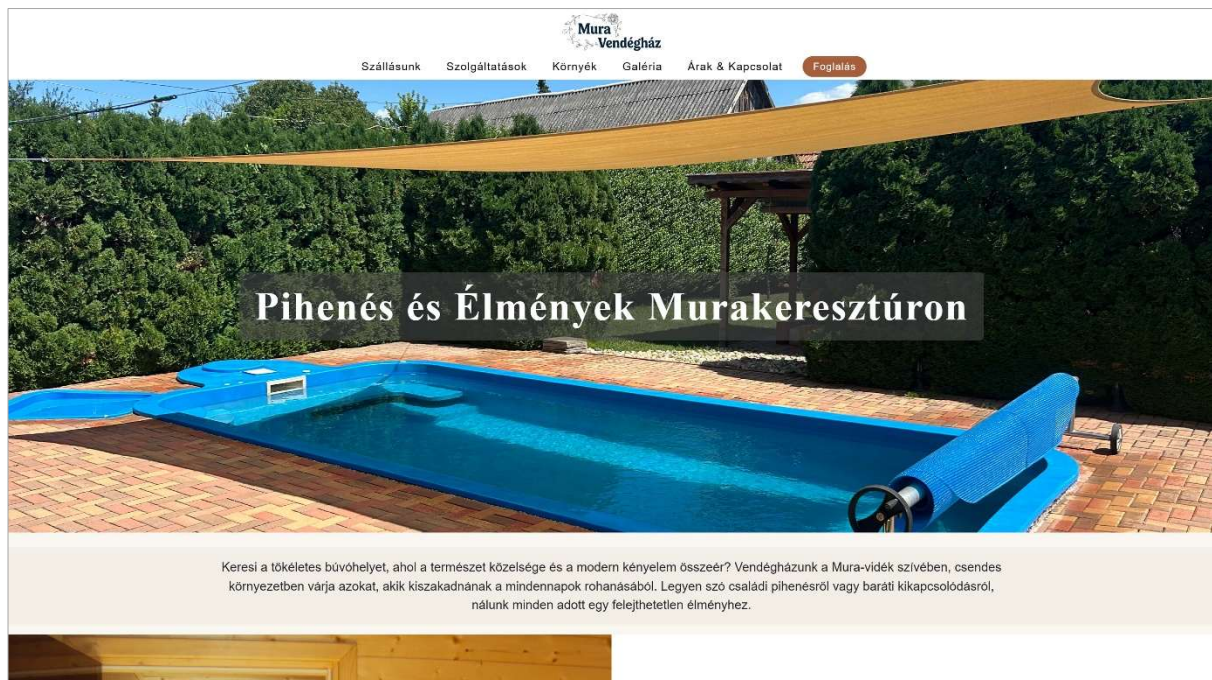
### 2.1. Rendszerkövetelmények és hozzáférés:

A program futtatásához a felhasználó részéről telepítés nem szükséges, eléréséhez mindössze egy modern böngésző (Chrome, Firefox, Safari) és internetkapcsolat kell. A Bootstrap használatának köszönhetően a felület reszponzív, így mobilon, tableten és asztali számítógépen is egyaránt átlátható és egyszerűen kezelhető.

### 2.2. A publikus felület (Vendég nézet):

#### 2.2.1. Navigáció és információs oldalak bemutatása

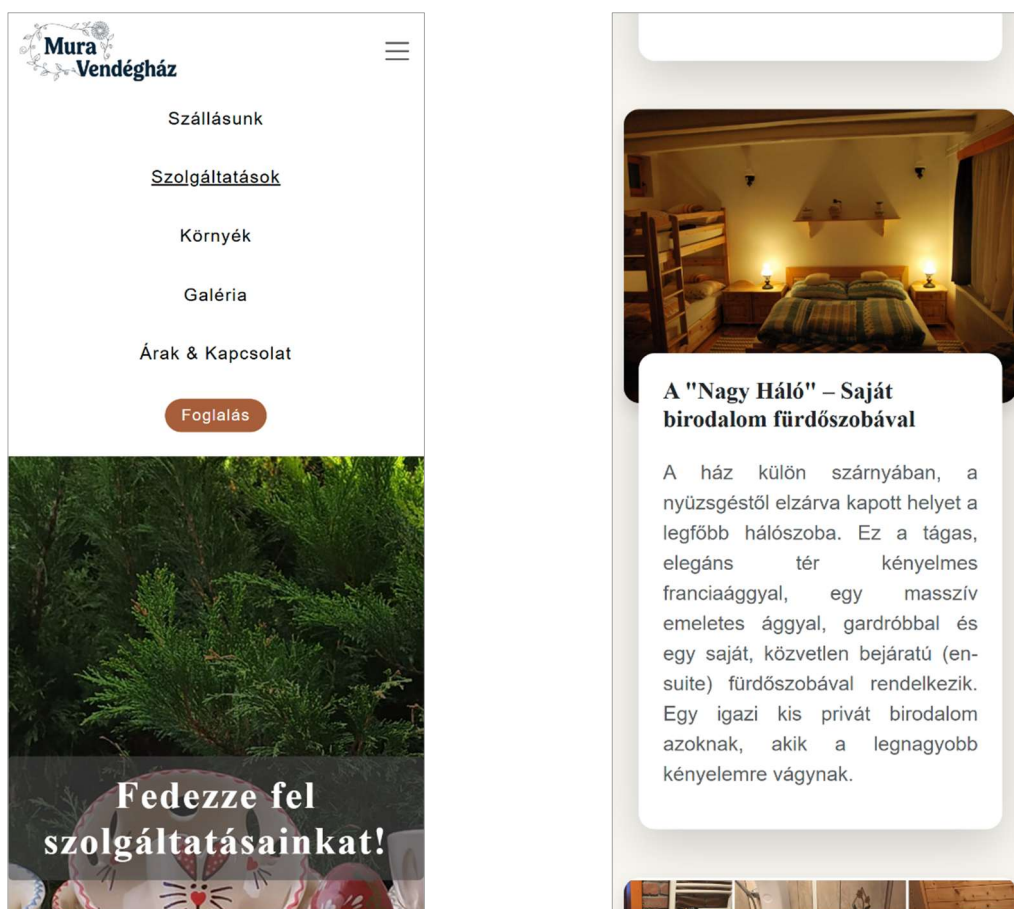
A weboldal publikus felületén fő szempont volt a logikus felépítés és a látogató végigvezetése az információkon. Ezt támogatja a menürendszer (1. ábra), amely megjelenik minden oldal tetején, és következetesen tagolja az egyes blokkokat. Ugyanígy, minden oldal tetején, a menüsor alatt, egy nagy nyitókép fogadja a látogatót, ami gyakori a modern webfejlesztésben, hiszen a profi hatás mellett egy jó kép bizalmat kelt a leendő vendégben.



1. ábra: A menüsor felépítése

A reszponzív tervezés lehetővé teszi azt, hogy mobilon, tableten, laptopon, és asztali számítógépen is egyaránt könnyen használható legyen az oldal. Amint a képernyő mérete egy bizonyos töréspont alá csökken, a weblap egyből reagál. A navigációs sáv ebben az esetben

úgynevezett „hamburger menüvé” változik, lenyitható füllé alakul, és a linkek egymás alá kerülnek, a többi kép és szövegdoboz szintén egymás alá rendeződik (2. ábra).



2. ábra: A Szolgáltatások és a Szállásunk menüpontok reszponzív, mobilos megjelenése

A felhasználói élmény a nyitóoldallal kezdődik. Ez fogadja a látogatót, amikor megnyitja a weboldalt. Ide nyitóképként a ház egyik fő attrakciója, a medence került. Lejjebb pár főbb szolgáltatást, és a kényelmet, nyugalmat kihangsúlyozó szövegek és képek mellett az alapvető tulajdonságok olvashatók. Az oldal alján a bizalmat tovább fokozó tapasztalatokról és értékelésekről esik szó.

A *Szállásunk* és *Szolgáltatások* oldalon bemutatásra kerül a ház belső és külső, udvar része. Előbbin szó esik a szobákról, és azok felszereltségéről, továbbá megtekinthető a ház alaprajza, és azok az alapvető belső tulajdonságai, amik alapján a felhasználók eldönthetik, hogy leadják-e a foglalást. Utóbbin a medence, szauna, a nyári garázs, a fedett sütőgető, és a kisgyermekeknek kedvező szolgáltatások kerülnek bemutatásra.

A Környék oldal a falut hívatott bemutatni, ahol a vendégház elhelyezkedik. Betekintést nyújt azokba a programokba, amiket egy pár perces sétával, vagy egy maximum 45 perces autós utazással elérhet.

Végül az *Árak & Kapcsolat* menüpontra kattintva a látogató megtekintheti az összes fizetős szolgáltatás árát kategóriákba szedve. Emellett a foglalással kapcsolatos gyakran ismételt kérdéseket végigolvasva információkat szerezhet többek között az előleg kifizetésének menetéről, a kiemelt időszakokról és az érkezéssel-távozással kapcsolatos tudnivalókról. Legalul megtekinthetők a tulajdonos elérhetőségei, és egy látványos űrlapon direkt üzenet is küldhető a weboldal üzemeltetőjének (3. ábra).

3. ábra: Kapcsolatfelvételi űrlap az *Árak & Kapcsolat* oldalon

#### 2.2.2. A foglalás menete

Miután a vendég átlapozta a különböző oldalakat, és eldöntötte, hogy foglalni szeretne egy időpontot, többféleképpen is eljuthat a foglalási űrlaphoz. A menüsorban kiemelt gomb jelzi a foglalást, ezen felül pedig mindegyik oldal alján egy cselekvésre felhívó sáv jelzi a benne lévő foglaló gombot (3. ábra). Miután rákattint valamelyikre, az űrlap fogadja, amin ha bal oldalt hiánytalanul kitölti a látogató a foglalás, és személyes adatait, akkor ez után – nagy képernyőn való böngészés esetén – jobb oldalon, az összesítés alatt aktiválódik a foglalás gomb (4. ábra).

**Foglalás igénylése**  
Az ár az időtartamtól függően változik.

**Légkondicionálás**  
Hűtés és komfort (+2.000 Ft/nap).  
A teljes időtartamra számolva, tájékoztató jellegű. Ha csak bizonyos napokra kéri, a helyszínen módosítható!

**Helyszínen igényelhető extrák**

- Szauna használat: 3.000 Ft / óra (A felfűtéstől számítva.)
- Fazekas bemutatás: 15.000 Ft / óra (Korongozás szakemberrel, anyaggal.)
- Sátározás: 6.000 Ft / fő / éj (Az udvaron.)

*Kattints a teljes árlistáért és az egyéb tudnivalókért!*

**Személyes adatok**

Teljes név \*  
Kovács János ✓

Telefonszám \*  
+234 ❗  
Kérjük, adjon meg egy érvényes telefonszámot!

Email cím \*  
kovacsjanos@email.com

Megjegyzés / Különleges kérés  
Érkezés várható ideje, étetőszek igény...

☐ Kijelentem, hogy elolvastam és elfogadom az Adatkezelési Tájékoztatót és a Házi rendet.

**Foglalás összesítése**

Időtartam **8 éjszaka**  
2026. 05. 21. -> 2026. 05. 29.

Szállásdíj (3 felnőtt) **288.000 Ft**  
Gyermek ár (3 fő) **144.000 Ft**  
Klíma díja (8 nap) **16.000 Ft**

Várható végösszeg **448.000 Ft**

\* A végleges fizetendő összeg a helyszínen kért extra szolgáltatások függvényében változhat. A foglalás elküldése még nem jár fizetési kötelezettséggel!

**Foglalás véglegesítése**

4. ábra: A foglalási űrlap befejezetlenül

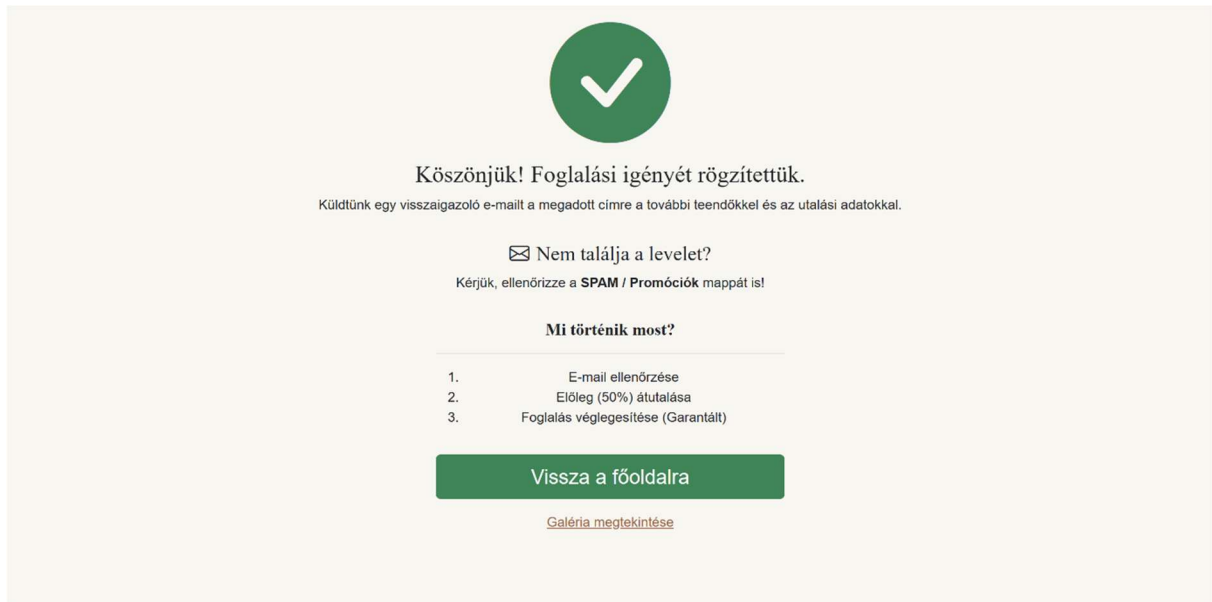
A foglalás leadása egy logikusan felépített űrlapon történik, amelynek lépései a következők:

1. Érkezés és távozás dátumának kiválasztása
2. Felnőttek és gyermekek számának beállítása
3. Kényelmi szolgáltatások (fűtés, klíma) igénylésének kiválasztása
4. Személyes adatok megadása (név, telefonszám, email)
5. Megjegyzés hozzáadása (opcionális)
6. Adatvédelmi tájékoztató és házirend elfogadása
7. Foglalás véglegesítése

Miközben a foglalás történik, a felhasználó végig nyomon követheti a várható végösszeg alakulását a *Foglalás összesítése* kártyán (4. ábra), ami valós időben, az oldal újratöltése nélkül reagál a bemeneti adatokra. Itt részletesen megjelennek a foglalással kapcsolatos információk, mint az éjszakák száma, illetve az érkezés és távozás dátuma, illetve lebontja részösszegekre a várható végösszeget.

A **Foglalás véglegesítése** gomb abban az esetben aktiválódik, ha a személyes adatok és a foglalás részletei sem akadnak fenn a kliensoldali validáción, tehát hiánytalanul és helyesen ki vannak töltve. Sikeres foglalás esetén a rendszer átirányítja a vendéget a *Sikeres foglalás* oldalra (5. ábra), amely tájékoztatja a foglalót a foglalás sikerességének tényéről, a következő

teendőkről, és lehetőséget nyújt visszalépni a főoldalra, vagy a galériába. Ez az oldal csak akkor elérhető, ha sikeres a foglalás, a link kézi beírása a keresőbe a foglalás oldalra fog irányítani. Emellett a háttérben kiküldésre kerül egy visszaigazoló email a vendégnek.



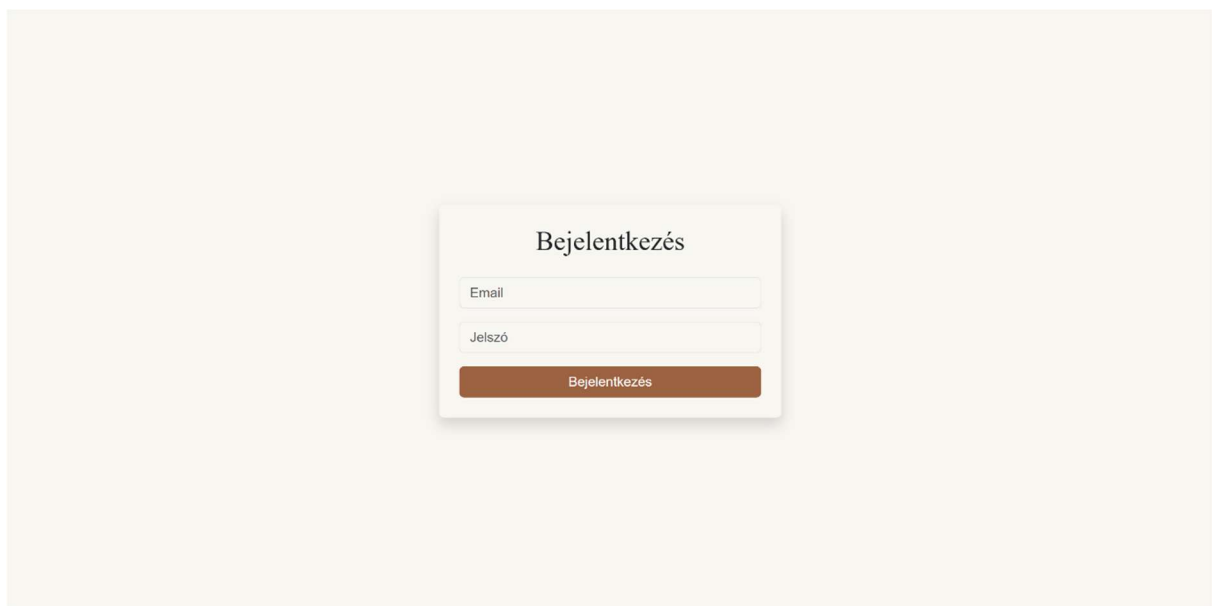
5. ábra: Sikeres foglalás oldala

## 2.3. Az Adminisztrációs felület bemutatása

A weboldal üzemeltetőjének szüksége van egy olyan zárt felületre, amely lehetővé teszi a dinamikusan változó adatok kezelését, mint például az árak, vagy a galéria képei. Ezen felül itt láthatja lista és naptár nézetben is a beérkezett foglalásokat. A felület segít a foglalások számontartásában, a foglalás és a foglaló adatainak módosításában, belső megjegyzések elhelyezésében. Akárcsak a publikus oldalon, itt is fontos szempont a reszponzív megjelenítés, hiszen gyakran telefonról szeretné kezelni a tulajdonos a weboldalt.

### 2.3.1. Hitelesítés és biztonság

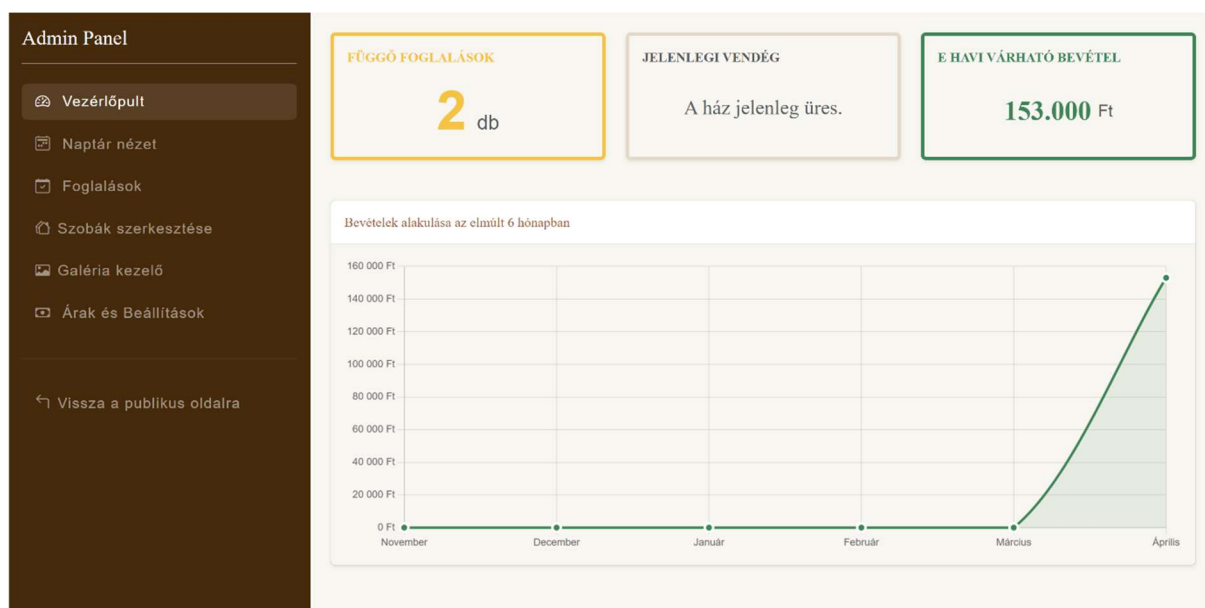
A nyilvános oldalon megjelenő adatok kezelésére szolgáló adminisztrációs felület nem publikus, eléréséhez megfelelő jogosultságok szükségesek. Mindez email címhez és jelszóhoz van kötve, amit a belépő oldalon adhat meg az üzemeltető (6. ábra). Ennek létrehozásához a Laravel beépített autentikációs rendszerét használtam.



6. ábra: Bejelentkező felület az adminisztrációs oldalhoz

### 2.3.2. Vezérlőpult és Foglalások kezelése

Belépés után az adminisztrátort a *Vezérlőpult (Dashboard)* fogadja (7. ábra). Itt megtekintheti a függő foglalások számát, a jelenlegi vendégeket, és az e havi várható bevételt. Az utóbbi hat hónap bevételének szemléltetését egy vonaldiagram is segíti, amelyet a Chart.js [5] integráció biztosít.



7. ábra: Adminisztrációs oldal vezérlőpultja

A vezérlőpulton a **Függő foglalások** kártyára nyomva, vagy a menüsoron a *Foglalások* pontot megnyitva egy átlátható, letisztult táblázatban láthatja a tulajdonos a jelenlegi foglalásokat. Megtekintheti a vendég adatbázis által generált azonosítóját (ID), nevét és telefonszámát, a foglalás intervallumát, a felnőttek és gyermekek számát, a várható végösszeget, és a foglalás státuszát (8. ábra). A táblázat *Műveletek* oszlopában két ikon található: a **Megtekintés** (szem ikon) gombra kattintva a foglalás részletei tekinthetők meg, míg a **Törlés** (kuka ikon) a rekord törlésére szolgál.

The 'Foglalások kezelése' section includes a 'Naptár nézet' (Calendar view) button and two tabs: 'Aktív foglalások' (Active bookings) and 'Lomtár (Törölt foglalások)' (Trash (Deleted bookings)). The active bookings are listed in the following table:

| #ID | Vendég neve                   | Dátum (Érk. - Táv.)          | Fő  | Összeg     | Státusz       | Műveletek |
|-----|-------------------------------|------------------------------|-----|------------|---------------|-----------|
| #3  | Takács Attila<br>+36301212121 | 2026.04.10. -<br>2026.04.13. | 2 2 | 117.000 Ft | Függőben      |           |
| #2  | Pán Péter<br>+36808808800     | 2026.05.13. -<br>2026.05.16. | 2 4 | 153.000 Ft | Visszalgazolt |           |
| #1  | Kovács János<br>+36301234567  | 2026.05.21. -<br>2026.05.29. | 3 3 | 448.000 Ft | Függőben      |           |

8. ábra: Foglalások felület az adminisztrációs oldalon

Utóbbit választva az adott sor eltűnik az aktív foglalások listájából, viszont nem törlődik végleg, így nem sérül az adatintegritás, és későbbi könyvvizsgálati (*audit*), analitikai, vagy jogi/vitarendezési folyamatok sem lesznek ellehetetlenítve. Szükség esetén a véletlenül törölt adatok a **Lomtár (Törölt foglalások)** szűrővel megtekinthetők és bármikor helyreállíthatók, ezzel növelve a rendszer hibatűrését. A **Megtekintés** műveletet választva külön ablakban, részletesen megjelennek az adott foglalás adatai (9. ábra). Itt az adminisztrátornak lehetősége van szerkeszteni a vendég személyes adatait, a foglalás paramétereit, illetve belső megjegyzést fűzhet a foglaláshoz.

9. ábra: Foglalás kezelése

Az adminisztrátornak továbbá módjában áll naptár nézetben megtekinteni a foglalásokat az erre dedikált *Naptár nézet* menüpontban. Itt átláthatóan, heti vagy havi lebontásban, de akár lista formájában is láthatja az érkező és távozó vendégeket.

### 2.3.3. Globális beállítások és árak dinamikus kezelése

A menüsorban az *Árak és Beállítások* menüpontra navigálva a tulajdonos egy egyszerű, felhasználóbarát űrlapon keresztül bármikor módosíthatja a dinamikus változó adatokat, mint a felnőtt és gyermek árak, fűtési felár, vagy a klíma, nappali vendég, rugalmas érkezés és szauna díja. Beállíthatja még a klíma opció elérhetőségét, és a kötelező fűtési szezont is, mindemellett átírhatja az oldalon megjelenő, a rendszer és tulajdonos által használt email címet és telefonszámot (10. ábra). Miután az adminisztrátor elmentette a módosításokat, a

weboldal automatikusan az új értékeket mutatja, és a foglalási folyamatban is ezek kerülnek felhasználásra, így nincs szükség a forráskód átírására.

The image shows two side-by-side screenshots of a mobile application's admin interface. The left screenshot, titled 'Admin Panel', displays 'Felnőtt árak' (Adult prices) with five input fields for different group sizes: '1 fő' (15000 Ft/éj), '2-3 fő' (12000 Ft/fő/éj), '4-5 fő' (10000 Ft/fő/éj), '6-7 fő' (9000 Ft/fő/éj), and '8+ fő' (8000 Ft/fő/éj). Below these is an 'Egyéb' (Other) section with a field for 'Gyermekeknek' (6000 Ft/fő/éj). The right screenshot, titled 'Globális Rendszerbeállítások' (Global System Settings), contains three toggle switches: 'Klíma opció aktiv.' (Climate option active), 'A weboldalon megjelenő kapcsolattartási email cím.' (Contact email on website), and 'A weboldalon megjelenő telefonszám.' (Phone number on website). Each toggle has a 'Bekapcsolva (Igen)' button and a technical key label. At the bottom is a green 'Beállítások mentése' (Save settings) button.

| Group Size | Price (Ft/éj) |
|------------|---------------|
| 1 fő       | 15000         |
| 2-3 fő     | 12000         |
| 4-5 fő     | 10000         |
| 6-7 fő     | 9000          |
| 8+ fő      | 8000          |

| Setting                                            | Value / Key                            |
|----------------------------------------------------|----------------------------------------|
| Klíma opció aktiv.                                 | Bekapcsolva (Igen) / ac_option_enabled |
| A weboldalon megjelenő kapcsolattartási email cím. | info@muravendeghaz.hu / contact_email  |
| A weboldalon megjelenő telefonszám.                | +36 30 123 4567 / contact_phone        |

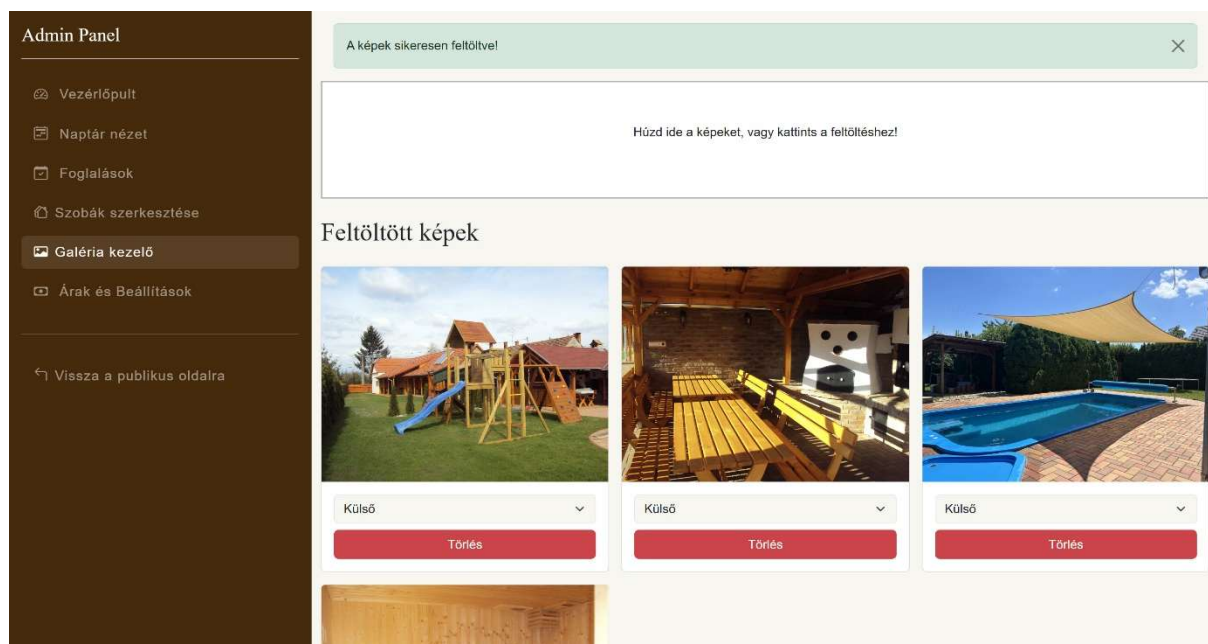
10. ábra: Árak és Beállítások módosítása mobil készüléken

#### 2.3.4. Tartalomkezelés (Galéria és Szobák)

A menüsoron a *Szobák szerkesztése* opciót kiválasztva az üzemeltető a publikus felület *Szállásunk* oldalán megjelenő szobák megnevezését és leírását láthatja. Egy integrált WYSIWYG (*What You See Is What You Get*, jelentése: amit látsz, azt kapod) szövegszerkesztő segítségével a szövegeket átírhatja, félkövérré vagy dőltté teheti, számozott és számozatlan listákat hozhat létre. Mindegyik szoba leírásához tartozik egy alap kép, ezt is módjában áll megváltoztatni. Az elmentett beállítások ebben az esetben is azonnal életbe lépnek a publikus felületen, és a dizájn igazodik a változtatásokhoz.

Az oldalsávon a *Galéria szerkesztése* (11. ábra) fül alatt kezelhetők a publikus oldal galériájában megjelenő képek. A tulajdonos itt tud hozzáadni és törölni fotókat, illetve kategorizálni a már feltöltött grafikákat. Egyszerre több kép is feltölthető, a kategorizálás pedig az oldal újratöltése nélkül megtörténik. Amennyiben sikeres volt a kategória

módosítása, a mező zöldre vált egy pillanatra, ellenkező esetben pirossal jelzi az eredménytelen változtatást.



11. ábra: Galéria szerkesztése oldal sikeres feltöltés után

### 3. Fejlesztői dokumentáció

#### 3.1. Felhasznált technológiák és fejlesztői környezet

##### 3.1.1. Fejlesztői környezet és verziókezelés

Verziókezeléshez a Git [6] nevű, nyílt forráskódú verziókezelő szoftvert használtam. A projekt létrehozásával párhuzamosan létrehoztam egy *MuraVendeghaz* nevű repository-t a GitHub nevű weboldalon. A rendszeres kódmódosítások (*commits*) és a távoli szerverre történő mentés (*push*) csökkentik az adatvesztés esélyét, lehetővé teszik az esetleges későbbi csapatmunkába való integrálást, és a *Conventional Commits* konvenció miatt átlátható, egyszerűen követhető és dokumentálható a haladási folyamat.

##### 3.1.2. Szerveroldali technológiák

A PHP egy általános szerveroldali szkriptnyelv dinamikus weblapok készítésére. Régre nyúló múltja miatt kiforrott ökoszisztéma övezi, így hatalmas közösség és rengeteg kész, tesztelt csomag (*package*) áll rendelkezésre.

A szakdolgozatom témájához tökéletesen illik az MVC (*Model-View-Controller*) architekturális minta, amelyet a Laravel nevű PHP keretrendszer használ. Ez a felelősségi körök szétválasztása (*Separation of Concerns*) elvet követi, amellyel az adatbázis logikája (Model), a felhasználói felület (View), és a műveletek irányítása (Controller) teljesen elkülönül egymástól [7]. Ezzel a megközelítéssel a kód könnyen követhető, letisztult marad, és egyszerű hibaelhárítást biztosít. A Laravel továbbá rendelkezik egy beépített adatbázis kezelő réteggel (*Eloquent ORM - Object-Relational Mapping*, magyarul Objektum-Relációs Leképzés). Ennek segítségével az adatbázis táblái PHP objektumokként kezelhetők, amellyel a lekérdezési logika külön, bonyolult SQL kódok nélkül is beintegrálható az üzleti logikába (*business logic*). A parancssori utasításokat az Artisan CLI (*Command-Line Interface*) alkalmazásával futtattam, ami egy, a Laravelhez tartozó parancssori felület, amely hozzájárul a mappafelépítés logikájának megőrzéséhez, és a kódgenerálás (*scaffolding*) segítségével szabványos, egységes felépítésű Controllereket, Modelleket hozhatunk létre.

Egy foglalási rendszernél, amely személyes adatokat is tárol, kritikus tényező a biztonság és a teljesítmény. A Laravel beépített SQL-injekció elleni védelemmel rendelkezik, amely a paraméterezett lekérdezésekkel megakadályozza a rosszindulatú kódok futtatását a beviteli mezőkön keresztül, így a felhasználótól származó adatok automatikusan tisztításra (*sanitize*)

kerülnek. Továbbá minden POST, PUT és DELETE kérésnél van CSRF (*Cross-Site Request Forgery*) védelem, amely egy automatikusan létrehozott tokenel azt hivatott ellenőrizni, hogy valóban a hitelesített felhasználó küldi-e a kérést. A teljesítmény növelésére pedig az aszinkron feladatkezelés szolgál, amely egy hosszú lefutású folyamat (például e-mail küldés) esetén nem várja meg a befejezést, ezzel megakasztva a felhasználói felületet, hanem a rendszer a háttérben elvégzi (*background processing*), ezzel növelve a felhasználói élményt (UX) és a szoftver hibatűrését (*fault tolerance*).

### 3.1.3. Adatbázis-kezelés

A MySQL egy széles körben elterjedt, megbízható Relációs Adatbázis-kezelő Rendszer (*Relational Database Management System, RDBMS*). A projektembe beleillik, hiszen tökéletesen integrálva van a Laravel környezettel. Egy foglalási rendszernél különösen fontos szempont az adatintegritás (*data integrity*) és a konzisztencia, amelyet a relációs modell biztosít a külső kulcsokkal és a tranzakciókkal, csökkentve a hibás rekordok létrejöttének esélyét. A rendszerben vannak egyaránt tranzakciós adatok, mint a foglalás, és konfigurációs táblák, mint az árak. A MySQL rugalmassága hatékonyan kezeli mindkettőt: teljesítményvesztés nélkül támogatja mind a szorosan csatolt (*tightly coupled*), mind a lazán csatolt (*loosely coupled*) táblákat. Az RDBMS struktúra kiváló skálázhatósága (*scalability*) jövőtűző tervezésre (*future-proofing*) is lehetőséget nyújtott, hiszen az adatbázis fel lett készítve egy esetleges későbbi továbbfejlesztésre, mint a foglalások kezelésének naplózása, vagy a különböző beállítások, képek egyedi sorrendezése.

### 3.1.4. Kliensoldali technológiák

A felület megjelenítésének struktúráját szemantikus HTML5 elemek, stílusát pedig CSS3 biztosítja. A tervezési fázisban kiemelt cél volt, hogy a weboldal bármilyen képernyőmérethez tökéletesen igazodjon, ezzel jelentősen megnövelve a felhasználói élményt.

A Bootstrap nevű CSS keretrendszer rácsrendszer alapú felépítése maradéktalanul kiszolgálja ezt az igényt, ezért erre esett a választás a fejlesztés során. A töréspontok (*breakpoints*) megadásával mobilon, tableten, és asztali számítógépen egyaránt személyre lehet szabni a rácsrendszer oszlopainak és sorainak elrendezését, ezzel eleget téve a *Mobile-first* paradigmának. Böngészőfüggetlen (*cross-browser*), és beépített segédosztályaival (*utility classes*) gyors prototipizálást (*rapid prototyping*) tesz lehetővé.

A programban a dinamikus interakciókhoz natív (*Vanilla*) JavaScriptet használtam, csökkentve ezzel a szerver felesleges terhelését.

### 3.1.5. Felhasználói felület és prototipizálás

A fejlesztés megkezdése előtt elengedhetetlennek láttam elkészíteni egy olyan UI/UX látványtervet, amelyet a végleges projektemnek elképzeltem, és amit végig szem előtt lehet tartani. Erre a célra a Figma [8] nevű, iparági standardnak számító felülettervező webalkalmazást választottam. Az elkészített drótvázak (*wireframes*) és a kattintható prototípus – bár a fejlesztés során agilis módon iteratív finomítások történtek – iránymutatóként vizualizálta számomra a követelményeket, ami jelentős mértékben gyorsította a fejlesztési folyamatot. A terv és a kész alkalmazás közötti összefüggést a 12. ábra is jól szemlélteti.



12. ábra: A prototípus (bal oldali) és a kész weboldal (jobb oldali) összehasonlítása

## 3.2. Rendszerarchitektúra és Üzleti logika

### 3.2.1. Az MVC architektúrális minta a gyakorlatban

A rendszer a Laravel által használt *Model-View-Controller* architektúrális mintára épül, amellyel garantálja a felelősségi körök szétválasztását, így átlátható, logikus mappastruktúrát eredményez. A projektemben a következő modellek felelnek az adatbázis-kapcsolatért és az *Eloquent ORM* relációk definícióiért: Booking, GalleryImage, Guest, PriceItem, RoomContent, SiteSettings, User.

A felhasználói felület a Laravel Blade sablonjaiból áll, ami biztosítja a Controller és a View közötti egyszerű kapcsolattartást. Más sablonkezelő rendszerekkel ellentétben a Blade nem korlátozza a tiszta PHP kód használatát a kódban, sőt, az egészet PHP-ra fordítja, és a cache-ben tárolja a módosításig, így kevésbé terheli a rendszert. A publikus és az adminisztrációs

felület Blade fájljai szintén követik a felelősségi körök szétválasztása elvet a biztonság és az átláthatóság érdekében.

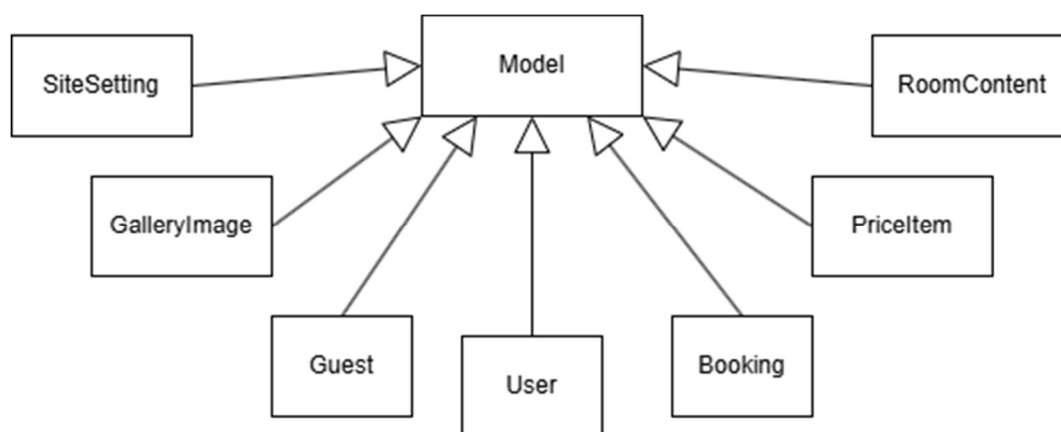
A Model és a View közötti kapcsolat a Vezérlők által jön létre. Ezek feladata a HTTP-kérések fogadása, a kapott bemeneti adatok validálása és a megfelelő HTTP-válasz (*response*) visszaküldése a kliens (böngésző) számára. A bonyolult számítások végzésére szolgál a Szolgáltatás (*Service*) réteg, a felelősségi körök itt is gondosan szétválnak.

### 3.2.2. Osztálystruktúra és UML modellezés

Az elkészült program alapjául szolgáló Az MVC architektúra objektumorientált (OOP) paradigmára épül. Ezen rendszerekben a kód tisztaságát és átláthatóságát az osztályok hierarchikus felépítése adja. Az egyes Modellek és Vezérlők esetében is van egy-egy központosított ősosztály (base class), amelyből az általam készített osztályok származnak. A közös, ismétlődő funkciókat (pl. adatbázis-műveletek, kérések kezelése) az ősosztály valósítja meg, ezzel követi a rendszer a DRY (Don't Repeat Yourself) elvet [9], ami a redundáns kód kiküszöbölésére szolgál.

#### 3.2.2.1. Adatmodellek (Models) hierarchiája

A Laravel az adatbázis-kezelést az Eloquent ORM rendszeren keresztül valósítja meg, ami az *Active Record* tervezési mintát követi. Ez azt eredményezi, hogy a keretrendszer beépített alaposztálya – Laravel esetében a `Illuminate\Database\Eloquent\Model` – az adatbázis táblákat és rekordokat objektumokba ágyazza, így minden ősosztálybeli metódus (`save()`, `find()`, `create()`, `delete()`) elérhető az egyes leszármaztatott modellekben. Ezzel a megoldással a projektben létrehozott adatmodellek nagyon vékonyak és tiszták maradtak, mert csak a táblák közötti kapcsolatokat és az egyedi szabályokat tartalmazzák. Az Eloquent Model ősosztály és a kész alkalmazás által felhasznált adatmodellek egyirányú leszármazási kapcsolatait a 13. ábra szemlélteti.

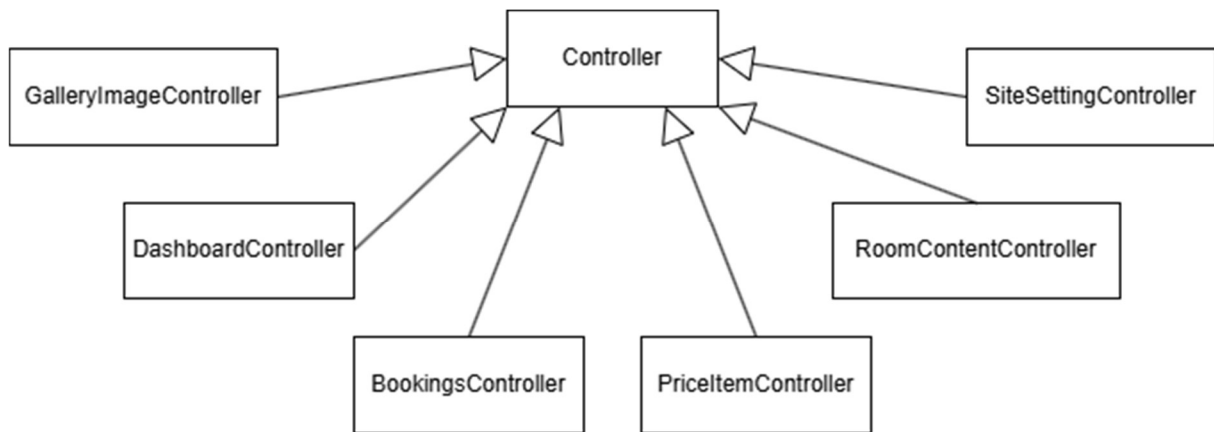


13. ábra: Az Eloquent Model ősosztály és a kész alkalmazás által felhasznált adatmodellek egyirányú leszármazási kapcsolatai

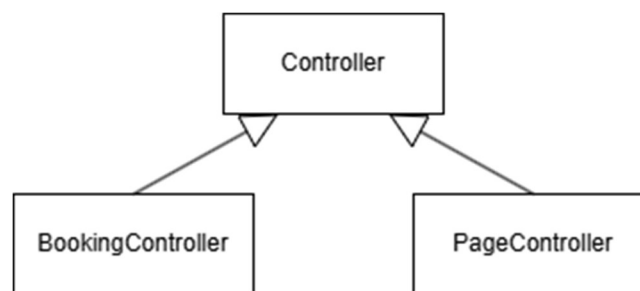
### 3.2.2.2. A Vezérlők (Controllers) öröklődési lánc

A Laravel a bejövő HTTP kérések és a rendszer válaszát irányító Vezérlő ősosztályaként az `App\Http\Controllers\Controller` osztályt használja. Ez olyan keretrendszer-szintű alapfunkciókat biztosít, mint a köztesrétegek (middleware-ek) hozzárendelése vagy a válaszok (pl. JSON vagy HTML nézetek) generálása. A fejlesztés során elvettem a klasszikus „minden táblának saját Controllere van” anti-mintát, helyette funkció-vezérelt Vezérlőket terveztem, így megőrizve a táblák közti magas (erős) kohéziót, és elkerülve a túlhízott vezérlő (*controller bloat*) jelenséget.

A rendszer vezérlői a már említett `Controller` ősosztályból örökölnék, így csak a nézeteknek való adatátadás, a kérések fogadása, illetve a beérkező adatok validálása és kezelése marad az általam elkészített vezérlők feladata. Az adminisztrációs oldalak Controllerei ezen alaposztályból származnak, de más köztesréteget használnak a hitelesítés érdekében, mint a publikus oldalak. A két felület Vezérlőit a felelősségi körök szétválasztása elvet követve különválasztottam egymástól. Az alábbi UML osztálydiagramok az adminisztrációs (14. ábra) és a publikus (15. ábra) modulok hierarchikus felépítését mutatják.



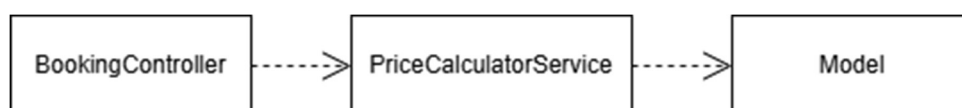
14. ábra: Adminisztrációs felület Vezérlőinek hierarchikus felépítése



15. ábra: Publikus felület Vezérlőinek hierarchikus felépítése

### 3.2.2.3. A Szolgáltatások (Services)

Az ár kalkuláció üzleti logikája során rengeteg különböző adatot kell ellenőrizni, mint például, hogy szabad-e az időpont, mennyi a jelenlegi felnőtt ár, vagy, hogy kér-e klímát a vendég. Erre bevezettem a Szolgáltatás réteget, ezzel biztosítva azt, hogy a Vezérlő megmaradt kizárólag a HTTP kérések fogadására és delegálására. A komplex üzleti logika Service rétegen belüli `PriceCalculatorService.php` fájlba való izolálása hozzájárul az ár kalkulátor későbbi, egyszerű felhasználásához kód duplikáció nélkül, amely a DRY elvet követi, és a felelősségi körök szétválasztása elvén túl a tartományvezérelt tervezés (*Domain Driven Design*) [10] módszertannak is megfelel, ráadásul a hibakeresés szintén drasztikusan leegyszerűsödött (16. ábra).



16. ábra: BookingController, PriceCalculatorService és a Model kapcsolatai

### 3.2.3. Az üzleti logika

Az üzleti logikát képző árszámítás egy külön PHP fájlban kapott helyet, számos manuálisan létrehozott tömbbel és változóval tesztelve a működését. A tartózkodási idő (éjszakák száma) meghatározásához a PHP Carbon (*DateTime API wrapper*) objektumorientált időkezelő könyvtárát alkalmaztam (`diffInDays()` metódus). A dátumok szimpla karakterláncként való tárolása rengeteg későbbi hibához vezethetne a szélsőséges eseteknél (*edge cases*), mint szökőév, vagy a hónap- és évváltások. Az éles projektben már dinamikus, Eloquent ORM alapú adatbázis-lekérdezések váltották a kódba égetett (*hardcoded*) árakat, ezzel lehetőséget adva az adminisztrátornak az adatok grafikus felhasználói felületen (*GUI*) keresztül történő valós idejű módosításának. Minden lekérdezésnél az éppen aktuális árakat implementáltam alapértelmezett (*fallback*) értéként, ezzel biztosítva a program ezen részének folyamatos működését előreláthatatlan körülmények (például helytelen bevételek) között is. Ezt hívják defenzív programozásnak (*Defensive Programming*) [11].

### 3.2.4. Dinamikus árkalkuláció

A publikus weboldal *Foglalás* menüpontja alatt található a foglalási űrlap, ahol a felhasználó megadhatja a dátumintervallumot, a felnőttek és gyermekek számát, és – az adminisztrátori beállításoktól függően – a klíma és fűtés igényét. Ezen adatok alapján történik az árkalkuláció, ami egy összetett folyamat, és valós időben reagál a felhasználói bemeneti paraméterekre.

A kalkulátor legelőször az érkezés és távozás dátumát kezeli. A Carbon segítségével megtörténik a parszolás, majd a `diffInDays()` metódus segítségével meghatározzuk az intervallum hosszát. A hibátűrés növelése érdekében – bár a kliensoldal is korlátozza a hibás bevitelt, – ismételt validáción esik át a kiszámolt éjszakák száma, hisz nem lehet kisebb, vagy egyenlő nullával, illetve minden újraszámolásnál ellenőrzésre kerül az is, hogy időközben nem foglalták-e le a vendégházat. Amennyiben valamelyik megtörténik, a számolás félbeszakad, kiírásra kerül a hiba.

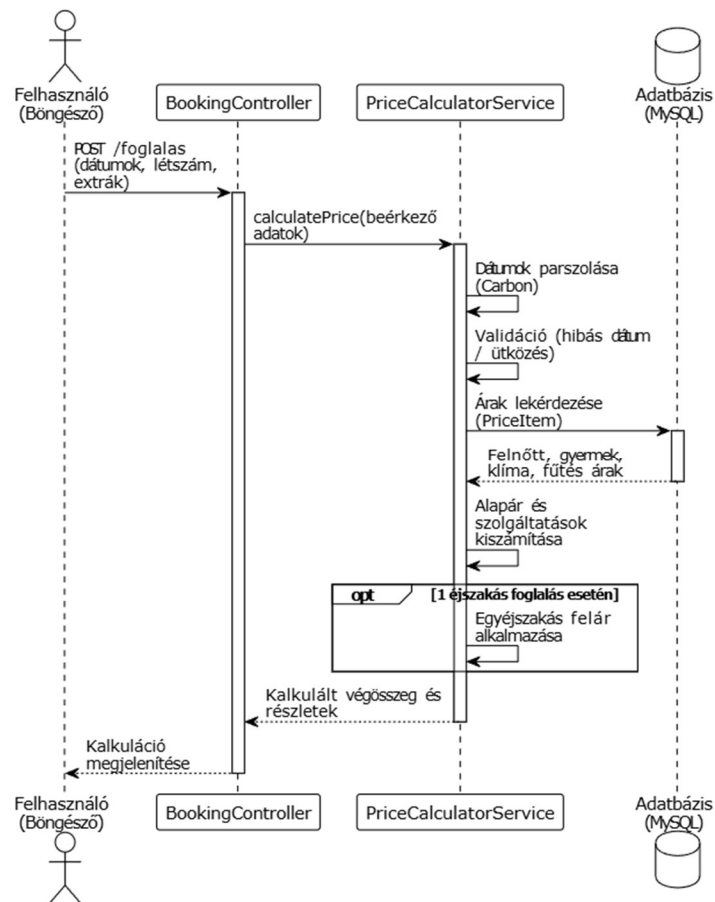
Ha minden rendben a dátumokkal, akkor következik a felnőtt árak kiszámolása, amely esetében a szállásadó hely sávós árazást alkalmaz. A `PriceItem` nevű táblából a `name` attribútumot használva a program adatbázis-lekérdezést indít a foglalásban szereplő felnőttek számához tartozó árat keresve, és elmenti egy változóba. Ugyanez az eljárás ismétlődik meg a gyermek és klíma árral, és a fűtési felárral is. Mindegyik esetében a fejlesztés idejében éppen

aktuális ár lett alapértelmezett értéként beállítva. Ezt követően az éjszakák száma, az előbbi lépésben megtalált ár, illetve a gyermekek és felnőttek száma alapján a rendszer meghatározza ezutóbbi két komponens végösszegét.

Következő lépésben az előző folyamathoz hasonlóan az éjszakák számával szorozva a fűtési felárat és a klíma díját számolja a rendszer, amennyiben elérhető és/vagy opcionálisok, továbbá igényli a felhasználó valamelyiket. A tulajdonosnak ugyanis lehetősége van a kötelező fűtési szezont bekapcsolni az adminisztrációs felületen, amivel az alapesettől eltérően, a foglalónak nem áll módjában maga választani, hogy szeretne-e fűtést, emellett megadhatja azt is, hogy a klíma opció elérhető legyen-e az oldalon.

A végösszeg e négy attribútum (felnőtt, gyermek, fűtés, klíma) összegéből jön ki. Utolsó lépésben a program megvizsgálja, hogy csak egyetlen éjszakát szeretne-e maradni a vendég, és amennyiben igen, minden részösszegre, és a végösszegre is alkalmazza az egyéjszakás felárat.

A kalkuláció pontos időbeli lefolyását a 17. ábra szekvenciadiagramja szemlélteti.



17. ábra: Az ár kalkuláció pontos időbeli lefolyása

#### 3.2.4.1. Az aszinkron adatáramlás és az élő frissítés megvalósítása

A hagyományos módszerrel ellentétben, ahol a **Számolás** gombra való rákattintást követően újratölt az oldal, és megjelenik az ár, az általam elkészített program dinamikus oldalfrissítést használva javítja a felhasználói élményt.

A frontend és a backend közötti kommunikáció Vanilla JavaScript (ES6) alapokon, a modern Fetch API segítségével valósult meg. Ez egy dedikált aszinkron REST végpontra küldi el az űrlap aktuális állapotát JSON formátumban. Ezt az egyirányú adatáramlást a Fetch API tökéletesen el tudta végezni.

Egy foglalás esetén a felhasználó hajlamos lehet egymás után gyorsan, több gombot is megnyomni. Ha minden egyes gombnyomáskor a böngésző azonnal küldene egy kérést, az a szerver és a hálózat túlterheltségéhez vezetne. Ennek elkerülése érdekében a *debouncing* tervezési mintát követtem: minden adatbevitel (*input*) vagy módosítás (*change*) során, amikor az eseményfigyelők aktiválódnak, 350 milliszekundumos késleltetést állítottam be. Ez az a

határ, ami még nem zökkenti ki a felhasználót, ugyanakkor drasztikusan csökkenti a felesleges hálózati terhelést (*network overhead*).

Fontos biztonsági szempont volt továbbá az is, hogy az ár kalkulációt ne a JavaScript végezze, hanem a backend. Amennyiben ez a kliensoldal dolga lenne, a felhasználó manuálisan belenyúlhatna a folyamatba, ezzel súlyos védelmi hibát hagyva a weboldalon. Azzal, hogy a számítást kizárólag a szerveroldali Service réteg végzi, garantáljuk az árak manipulálhatatlanságát és azt, hogy az üzleti logika egyetlen, ellenőrzött helyen (*Single Source of Truth*) létezzen [12].

### 3.2.5. Útválasztás (*Routing*) és Köztesrétegek (*Middleware*)

A Laravel a webalkalmazás végpontjait (*endpoints*) a `routes/web.php` fájlban definiálja. Ezen pontok eléréséhez különböző útvonalak hozhatók létre, amelyek alapján az egyes publikus és adminisztrációs oldalak megtalálhatók. A publikus oldalak (*Főoldal, Szállásunk*) egyszerű, bárki által kiadható GET kérésekkel (*HTTP GET requests*) működnek, míg az adminisztrációs oldalakhoz ideális esetben csak a tulajdonos férhet hozzá. A Köztesrétegek kényelmes eszközt nyújtanak az alkalmazásba beérkező HTTP-kérelmek ellenőrzéséhez és szűréséhez. Az általam használt, Laravelbe beépített `auth` (*authentication*) Middleware azt hivatott ellenőrizni, hogy az alkalmazás felhasználója hitelesítve van-e. Amennyiben ez nem teljesül, elfogja a kérést (*request interception*) és a rendszer átirányítja (*redirect*) a látogatót a bejelentkező felületre, így a védett végpontokhoz tartozó HTTP-kérések kizárólag hitelesített felhasználó esetén hajtódnak végre. Ennek hatékony használatára létrehoztam egy Köztesréteg csoportot (*Route Group*), majd az adminisztrációs felülethez tartozó összes végpontot ebben központosítottam. Ezzel a biztonság alapértelmezésként (*Secure by Default*) elvet [13] követve ahelyett, hogy a Controller minden egyes metódusában külön ellenőriznénk a munkamenet (*Session*) érvényességét, a Middleware még az üzleti logika elérése előtt elfogja a kérést. Ez a későbbiekben segít kiküszöbölni a fejlesztői mulasztásból adódó biztonsági réseket.

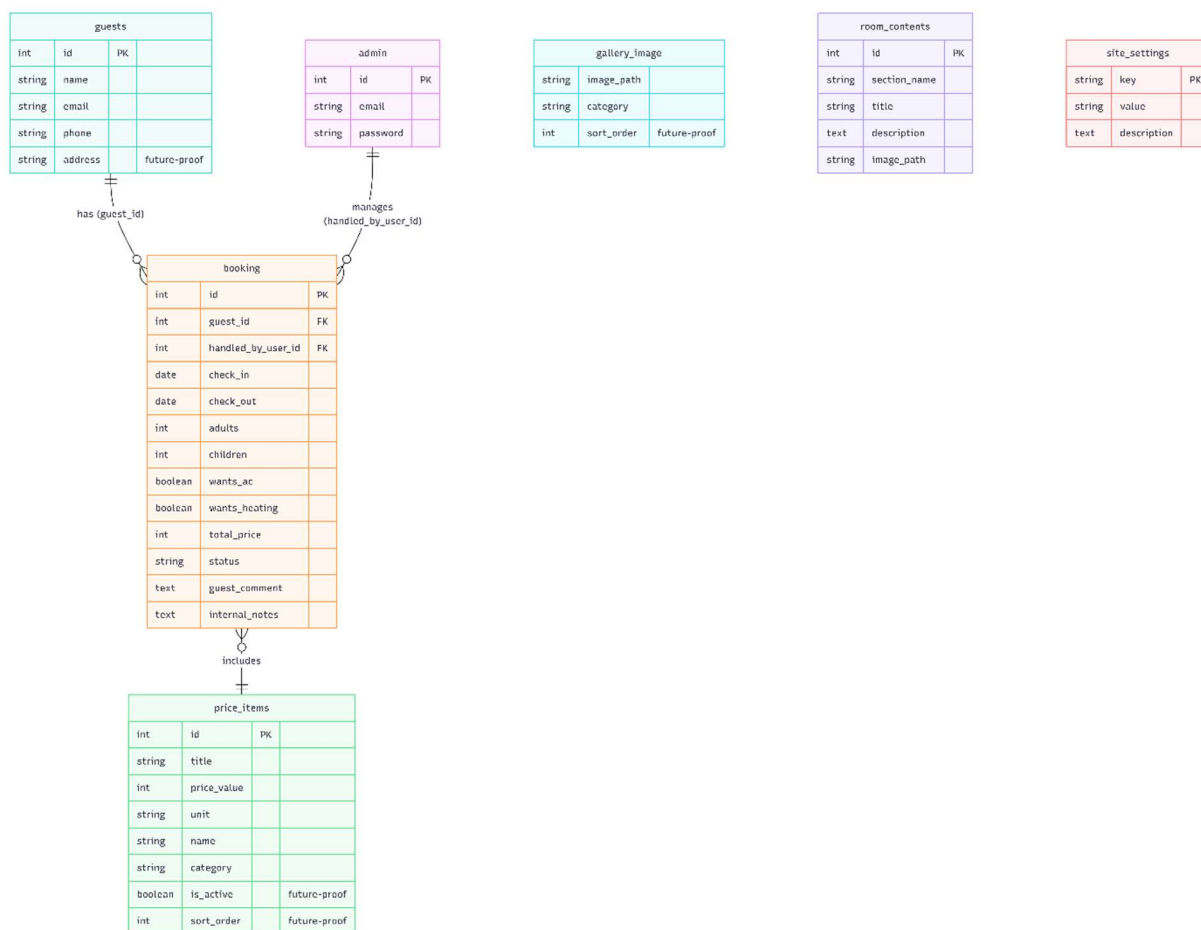
A Laravel `Route::resource()` szigorú, egy-entitásos (*single-entity*) RESTful végpontokat ad. Az adminisztrációs felületen az árak, globális beállítások, és szobák frissítésénél eltértem ezen elvektől, hiszen feleslegesen idő- és erőforrásigényes program jött volna létre, ha minden egyes tétel frissítése külön, egyéni hálózati kéréssel lenne megoldva. Ehelyett olyan egyedi

végpontot (*custom endpoint*) hoztam létre, ami egyetlen, payload-alapú iterációval tömegesen frissíti (*batch processing*) a rekordokat, ezzel a pragmatikus megközelítéssel előtérbe helyezve a felhasználói élményt és a hatékonyságot.

### 3.3. Adatbázis-tervezés és Adatkezelés

#### 3.3.1. Relációs adatmodell és ER-diagram

A projektemben a MySQL nevű, széles körben elterjedt, megbízható RDBMS rendszert használtam. Ez lehetővé tette az adatok izolálását és a táblák összekapcsolását egyaránt. Ez különösen fontos egy foglalási rendszernél, ahol a vendég hozza létre a foglalást, az admin pedig kezeli. A relációs modell megfelelő alkalmazása garantálja az adatok normalizálását, így elkerülhető a redundancia (például egy visszatérő vendég adatainak többszöri, felesleges letárolása), és biztosított a vendégek és foglalások független kezelhetősége. Ez a struktúra révén logikai kapcsolatok hozhatók létre az egyes táblák között – ezen kapcsolatokat remekül megmutatja a 18. ábra is, amelyen az elkészült projekt Egyed-Kapcsolat / *Entity-Relationship* (ER) diagramja látható.



18. ábra: Az elkészült projekt Egyed-Kapcsolat diagramja

Az ER diagramon szereplő `admin` táblát a beépített `users` tábla helyettesíti a kódban, ezzel megkönnyítve az `auth` middleware használatát az adminisztrációs felület hitelesítési logikájának integrálása során.

Két tábla között többféle kapcsolat is létrehozható. A `guest` (vendég) és a `booking` (foglalás) tábla között 1:N kapcsolatot definiáltam, hiszen egy vendégnek több foglalása is lehet, és ugyancsak 1:N kapcsolat van a `users` és a `booking` között is. Utóbbit a jövőtűző tervezés jegyében építettem a kódba, ezzel megkönnyítve azt a továbbfejlesztést, ahol több adminisztrátor is hozzáfér az adatokhoz, és naplózható, hogy melyiket, mikor, melyik módosította, kezelte.

### 3.3.2. Adatbázis-migrációk és az Eloquent ORM a gyakorlatban

Az adatbázis tábláinak létrehozását migrációs fájlok segítségével végeztem el. A Laravel ezzel a megoldással jelentősen leegyszerűsíti, sőt, garantálja a verziókövetést, ezen felül lehetővé teszi az azonnali visszavonást, és akár a teljesen tiszta lappal kezdést is.

A táblák létrehozásakor a jövőbeli javítások és hibák elkerülése érdekében mérlegelnem kellett néhány dolgot. Egyik ilyen volt az adattípus kiválasztása mennyiségeknél és áraknál. A mennyiséget vagy árat mutató adatoknál előjel nélküli egész (`unsignedInteger`) adattípust alkalmaztam, mert ezek nem vehetnek fel negatív értéket, ezzel is további, immár adatbázis-szintű védelmet is biztosítva a programomnak.

Továbbá a globális változóknál Kulcs-Érték (*Key-Value*) adattárolási mintát alkalmaztam. Így tehát például a fűtés kapcsoló, vagy a telefonszám nem egy saját oszloppal rendelkezik, vagy nem egyetlen sorból álló hagyományos táblában tárolódik, hanem a Key-Value struktúrájú `site_settings` táblában, ahol a `key` az elsődleges kulcs (*PRIMARY KEY*). Ez praktikus, ha új globális beállítást szeretnék a későbbiekben hozzáadni. Ennek megfelelő működéséhez a *SiteSetting* Model konfigurációjában is explicit módon felül kellett írnom a keretrendszer alapértelmezett viselkedését, máskülönben az ORM hibás SQL lekérdezéseket generált volna.

Mindemellett `price_items` és `gallery_images` táblákban definiáltam egy `sort_order` nevű attribútumot, ezzel szintén a jövőtűző tervezést támogattam. A későbbiekben ha egyik jobb képet előre szeretnék hozni az oldalon, vagy egy új árat veszek fel, és nem a lista végére szeretném, akkor ennek segítségével lesz rendezhető a felsorolás.

A jelenlegi, alapértelmezett adatok (árak, szobaleírások, beállítások, adminisztrátor bejelentkezési adatok) feltöltése a Laravel *Seeder* osztályával történt. Ez az infrastruktúra mint kód (*Infrastructure as Code*) elv [14] egy mikroszintű megnyilvánulása az adatbázis-kezelésben. A *Seeder* osztályokban a hagyományos `create()` vagy `insert()` metódusok helyett az *Eloquent* `firstOrCreate()` metódusát alkalmaztam, ezzel garantálva az idempotens működést. Ennek alkalmazásával a *Seeder* osztályokat felhasználó, terminálban futtatott parancs többszöri kiadásra kerülése esetén sem keletkeznek duplikált rekordok.

### 3.3.3. Adatintegritás, GDPR és a törlési stratégiák

A fejlesztés kezdeti fázisában az egyik fő szempont az árva rekordok kiküszöbölése volt, a technikai integritás jegyében. Ennek biztosítására a puha törlés (*Soft Delete*) szolgált. A Laravel keretrendszer *SoftDeletes* tulajdonságát (*trait*) alkalmazva a rekord nem törlődik véglegesen, csak kap egy `deleted_at` időbélyeget (*timestamp*). Ezzel nem sérül az adatintegritás, de egyúttal a globális hatókörök (*global scopes*) automatikus szűrése révén az

adminisztrációs felületen nem fognak megjelenni a vendég adatai (GDPR), de a foglalás rekordja továbbra is be tudja tölteni a pénzügyi szerepét az összesítésekben. Ráadásul a véletlenül törölt adatok a `restore()` metódussal bármikor helyreállíthatóak, növelve a rendszer hibatűrését.

### 3.4. Biztonsági Architektúra

#### 3.4.1. Adatbeviteli sebezhetőségek és jogosultságkezelés

A fejlesztés során különös figyelmet igényelt a biztonsági kockázatok számának minimalizálása, kiküszöbölése. A legkézenfekvőbb biztonsági rést az illetéktelen behatolók számára a bemeneti mezővel rendelkező űrlapok jelentik. Ennek elkerülésére a rendszerben modell-szintű védelem (*mass assignment*) szolgál. Annak megakadályozása érdekében, hogy a felhasználó olyan adatokat is beküldjön, amihez nincs joga, a Laravel modellek adatbázisba mentésénél a `$fillable` tömböt használtam a `$guarded` tömbbel szemben, mert előbbi a beérkező HTTP-kérésből kizárólag a benne definiált attribútumok számára engedélyezi a rögzítést. Utóbbi egy feketelista (*blacklist*), ami pedig a benne megtalálható adatokat védi. Ez egy jelentős biztonsági kockázatot jelent, ha később egy új fontos, érzékeny oszlop kerülne az adatbázisba, és a fejlesztő elmulasztja a `$guarded` tömb kibővítését, ami jogosultság-kiterjesztéses (*Privilege Escalation*) támadáshoz vezethet. A `$fillable` ezzel ellentétben a biztonság alapértelmezésként paradigmát követi, hisz eredetileg zárt, így egyetlen új mező sem írható felül explicit engedély nélkül.

Veszélyt jelent továbbá a feltöltött képeken keresztül érkező támadás. Ezek esetében nem elég a szimpla kiterjesztés-ellenőrzés, mert a fájl kiterjesztése könnyen módosítható, és akár egy kártékony parancsfájl (*web shell*) is lehet, ami később bárhonnan távoli kódfuttatással (*Remote Code Execution*) károsíthatja a rendszert. Ezért a program a fájl bináris fejlécét (*Magic Byte*) vizsgálja meg a MIME (*Multipurpose Internet Mail Extensions*) internetes szabvány `Content-Type` fejléce alapján. Ezen felül a maximális fájl méretnek 4 MB-os korlátot állítottam a tárhely betelésének (*Disk Exhaustion / DoS attack*) elkerülésére. A felhasználói élmény növelése érdekében a Drag-and-Drop funkcióval rendelkező Dropzone.js [15] komponenst alkalmaztam, amihez a biztonsági lépések is egyszerűen igazíthatók voltak.

Jelentős biztonsági kockázatot jelent továbbá az adminisztrációs felületen történő szobaszerkesztés. Erre a TinyMCE [16] nevű megbízható WYSIWYG szerkesztő könyvtárat

használja a program, ami nyers HTML kódot ment az adatbázisba. A Laravel alapértelmezetten kontextus-függő kódolással (szanitizálással) semlegesíti a bemeneti HTML kódokat. Ahhoz, hogy a tulajdonos által szerkesztett kód úgy jelenjen meg a publikus felületen, ahogy ő formázta, felül kellett írnom a Laravel ezen védelmi mechanizmusát, és nyers rendereléssel kiírni az elmentett szobaleírást. Amennyiben a publikus felületen egy hozzászólásokat, vendégértékeléseket fogadó felületen kellett volna formázott szöveget kiírni, ez a megoldás szigorúan tilos lett volna. Ebben az esetben egy harmadik féltől származó HTML Purifier (Tisztító) könyvtárat kellett volna alkalmazni, ami a formázó tageket bent hagyja, a szkript megadására alkalmasokat eltávolítja.

### 3.4.2. Többrétegű validáció és a Zéró Bizalom (*Zero Trust*) modell

A biztonsági architektúra tervezése során kritikus döntés volt a validációs rétegek elválasztása, mivel a pusztán kliensoldali ellenőrzés súlyos sebezhetőséget jelent.

A WYSIWYG (TinyMCE) szerkesztőmezőnél bevezettem egy 500 karakteres limitet. Ez kliensoldalon figyel, és a JavaScript `keydown` eseményfigyelőjével blokkolja a túllépést. Ez azonban csak a felhasználói élmény javítását szolgálja, megspórol pár feleslegesen kiküldött hálózati kérést, biztonsági szempontból viszont önmagában elégtelen (insufficient). Mivel a böngésző egy kliens által kontrollált környezet (client-controlled environment), a beérkező adatok redundáns ellenőrzése elengedhetetlen. Ennek érdekében szerveroldalon – a mélységi védelem (*Defense in Depth*) alapelv [17] alapján – a Laravel `Validator` osztályával redundáns ellenőrzés alá esnek a beérkező adatok, így megvédve a leírás oszlopának integritását.

A weboldal talán legfontosabb része az üzleti logika, az ár kalkuláció, hiszen egy pontatlan számolás esetén – bár jogilag nem feltétlenül köteles a szállásadó az irreálisan alacsony áron biztosítani a szolgáltatást – súlyos fogyasztóvédelmi aggályok, adminisztrátori többletmunka, és a vendég bizalmának elvesztése is felmerülhet. Ezért különös figyelem irányult a fejlesztés során a dinamikus ár kalkulátor aszinkron végpontjának megvédésére a manipulált kérésektől. Amikor érkezik egy JSON csomag a Controllerbe, a Vezérlő elveti az adatcsomag-megbízhatóság (*payload trusting*) hibás koncepcióját, nem bíz meg vakon a beérkező JSON csomagban, mert például egy rosszindulatú aktornak (*malicious actor*) lehetősége lenne kikerülni a kötelező fűtési szezont. Ehelyett minden foglalásnál az adatbázisból vannak

lekérdezve ezek az adatok. A legfontosabb része azonban a rögzítésnek az ár újraszámolása, amely minden alkalommal biztosítja azt, hogy pontos, az aktuális árak által képzett végösszeg kerüljön az adatbázisba. Ezt a rendszer a zéró bizalom (*Zero Trust*) modellt [18] alkalmazva valósítja meg: minden foglalás rögzítése előtt egy szerveroldali állapot-újraellenőrzést (*state revalidation*) hajt végre a biztonságos adatbázis-szabályok alapján. Ez garantálja, hogy a tranzakció pénzügyi integritása manipulálhatatlan maradjon.

### 3.4.3. Biztonságos HTTP kommunikáció és tranzakcióvédelem

A modern webalkalmazások biztonsága nemcsak az adatbázison, hanem a HTTP protokoll szabályainak szigorú betartásán is múlik.

A publikus *Foglalás* oldalon található *Foglalás összesítő* JavaScripttel (Fetch API) kommunikál a szerverrel. Bár a hagyományos, szerveroldalon renderelt (SSR) űrlapoknál a Laravel `@csrf` direktívája automatikusan megvéd a támadásoktól, az aszinkron kérések esetén a DOM-szintű JavaScript manuális tokenkezelést igényel. Ehhez a HTML `<head>` szekciójába generáltam egy `meta` taget, amit a JavaScript futásidőben beolvas, és egy egyedi HTTP fejlécbe (*Custom HTTP Header, X-CSRF-TOKEN*) injektál, ezzel garantálva, hogy egy idegen, kártékony weboldal ne tudjon hamisított keresztoldali kérést (*Cross-Site Request Forgery – CSRF*) indítani a gyanútlan felhasználó böngészőjéből [19].

További HTTP biztonsági kérdés a destruktív tranzakciók (*destructive transactions*) védelme. A webes HTTP szabványok szerint a `GET` kéréseknek szigorúan idempotensnek kell lenniük, ezzel garantálva az adatbázis változatlanságát bármennyi kérés küldése esetén. Amennyiben a törlés (például adminisztrációs felületen egy foglalás törlése) egy `GET` útvonalon lenne elérhető, az katasztrófához vezethetne. Egy, a weboldalt feltérképező keresőmotor-robot (Googlebot) vagy a böngészők oldal-előretöltő (*prefetching*) algoritmus a linkeket beolvasva véletlenül kitörölhetné a teljes adatbázist. Ennek elkerülésére a törlés gomb valójában egy `POST` kérést küldő űrlap, amit a Laravel metódus-hamisítással alakít át szabványos `DELETE` kéréssé, így biztonságosan, hálózati szinten CSRF tokennel és a felületen egy JavaScript megerősítő ablakkal (*confirmation dialog*) védve a műveletet.

### 3.5. Teljesítményoptimalizálás és Felhasználói Élmény (UX/UI)

#### 3.5.1. Backend teljesítmény és erőforrás-kezelés

A webalkalmazások teljesítményoptimalizálásának (*performance optimization*) két alappillére van: a szerveroldal adatelérésének és -továbbításának sebessége, illetve a kliensoldalnak szükséges erőforrások hatékony letöltése.

Az adminisztrációs felület gyorsasága a fejlesztés korai szakaszában nem volt megfelelő felhasználói élmény szempontjából. A foglalások listázása az ORM rendszerek gyakori hibájából adódóan lusta betöltéssel (*lazy loading*) érte el az adatbázisból az információkat. Ezzel a megoldással 15 foglalás esetén a rendszer indított egy lekérdezést a foglalásokra, majd további 15 iteratív SQL lekérdezést a hozzájuk tartozó vendégekre. Ez 15+1 lekérdezést eredményezett, ezzel megkapva az N+1 problémát. A teljesítmény növelésének érdekében ezt a logikát a Controllerben felülírtam előtöltésre (*eager loading*) a `Booking::with('guest')` szintaxissal. Ez azt eredményezi, hogy a keretrendszer mindössze 2 SQL kéréssel lekéri a teljes adathalmazt, ezzel – főként sok adat esetén – jelentős mértékben csökkentve az adatbázis-szerver I/O terhelését és a hálózati válaszidőt (*latency*).

További, a frontend betöltésének sebességét fokozó megoldás a letöltendő fájlok méretének csökkentése. Eredetileg NPM-ből érkező globális CSS-t használtam az ikonokhoz, azonban ezt teljesítmény-optimalizált, natív Blade SVG komponensekre cseréltem. A CSS és JavaScript betöltését a statikus `<link>` tagek helyett a modern Vite integrációval oldottam meg. Ez a fejlesztési munkának a gyorsaságát is megnövelte, hiszen minden mentésnél azonnali, moduláris frissítést (*Hot Module Replacement - HMR*) [20] biztosít, élesítéskor (*production build*) pedig Rollup technológiával drasztikusan összetömöríti a fájlokat, ezzel odafigyelve az oldal első bájtjának letöltéséig eltelt időre (*Time to First Byte – TTFB*) [21].

#### 3.5.2. Core Web Vitals és Frontend ergonómia

A modern webes ergonómia (*Core Web Vitals*) meghatározó szegmense a vizuális stabilitás (*Cumulative Layout Shift*) [22]. Gyakori hiba a webfejlesztésben az a jelenség, amikor a honlap betöltésekor a felület váratlan elmozdulása (*layout shift*) történik. Ezt leggyakrabban az okozza, hogy egy képnek nincsenek előre definiálva a méretei, és a böngésző renderelő motorja (*rendering engine*) nem tudja, mekkora helyet (*placeholder space*) foglaljon le neki. Először betölti az oldalt, majd a képeket, és azok méretei alapján újrarendezzi a már betöltött

tartalmat (DOM-újrarendeződés / *reflow*). Ezt kiküszöbölendő, minden kép esetében explicit módon deklaráltam a `width` és `height` attribútumokat, amit a Bootstrap `img-fluid` osztályával kombináltam. Ezzel a megoldással a böngésző már a letöltés előtt kiszámítja a képarányt (*aspect ratio*), és utána a Bootstrap segítségével a kép ahhoz a helyhez lesz igazítva, így javítva a UX-et és a SEO (keresőoptimalizálás) megítélést is.

A vizuális ergonómia másik kritikus pillére az, hogy a szöveg mindig olvasható maradjon a változó hátterek előtt. A publikus felület minden oldalán *Hero* szekciót alkalmaztam, ahol a háttérkép mindig eltérő. A ráhelyezett tipográfia olvashatóságát (*legibility*) megőrizve egy CSS-alapú, áttetsző sötétítő réteget helyeztem el a címsor mögött. Ezzel garantáltam, hogy a világos tipográfia mindig, minden körülmények között megfeleljen a WCAG (*Web Content Accessibility Guidelines*) [23] vizuális kontrasztarányra vonatkozó szigorú követelményeinek. Ráadásul, hogy egyszerűen, stílusban megegyező, és egységesen változtatható *Hero* szekciót kapjak, a kódredundanciát elkerülve a `layout` komponensben helyeztem el azt. A reszponzivitás megőrzésére, hogy a képek képarány-torzulás (*distortion*) nélkül jelenjenek meg bármilyen képernyőméreten, az `object-fit: cover` CSS tulajdonságot alkalmaztam.

### 3.5.3. Kognitív terhelés minimalizálása és Zökkenőmentes UX (Seamless UX)

#### 3.5.3.1. Vizuális hierarchia és Fókuszált Feladatvégzés

Az ember-gép interakció (HCI) során kiemelten fontos a felhasználó figyelmének irányítása és a zavaró tényezők kizárása.

A kognitív terhelés (*cognitive load*) minimalizálására kiváló példa a publikus főoldal kedvcsináló szekciója, ahol asztali nézetben cikk-cakkos (Kép-Szöveg, majd Szöveg-Kép) elrendezést használtam a kedvcsináló szekcióban. Ez megtöri a vizuális monotóniát, megelőzi a banner-vakságot (*banner blindness*), és a szemmozgás-követéses (*eye-tracking*) kutatásoknak megfelelően, Z-alakban (*Z-pattern*) vezeti végig a felhasználó szemét a tartalmon. Az így elkészült program mobilon alapértelmezetten szöveg-kép-kép-szöveg elrendezésben jelent meg. Ezt a HTML forráskód redundáns duplikálása nélkül CSS Flexbox `order-*` segítségével javítottam, aminek hatására lineáris sorrendbe kerültek az egyes blokkok.

A zavaró tényezők kizárása érvényesül az adminisztrációs felületre való *Bejelentkezés* oldalán. Mivel a hitelesítés egy zárt, magas fókuszú igénylő feladat, a navigációs sávot és a lábléceket szándékosan eltávolítottam, így az izolált *Login* platformról eltűntek a felesleges kijárat pontok (exit points), amivel a konverzióra (bejelentkezésre) fókuszálja az oldal a figyelmet.

#### 3.5.3.2. Hibamegelőzés és Vizuális Affordancia

A klasszikus foglalási rendszert használó weboldalak (anti-minta) gyakran hagyják, hogy a felhasználó kiválasszon egy már lefoglalt dátum intervallumot, és csak a véglegesítés gombra való kattintáskor, utólagos szerveroldali validáció (*post-validation*) során jelzik a hibát. Az általam készített program a Poka-Yoke (hibaelkerülés) elvet követi [24]. Az adatbázisból kinyert adatokat a szerver átadja a kliensoldalnak, és a benne szereplő, már lefoglalt napokat szürke háttérrel letiltja (*disabled*), ami egyértelmű negatív affordanciát (*negative affordance*) biztosít. Ez vizuálisan egyértelműsíti a rendszer állapotát anélkül, hogy a felhasználónak bármilyen interakcióba kéne lépnie az oldallal, így drasztikusan csökkentve az interakciós súrlódást (*friction*) és a kognitív terhelést. A kliensoldali naptár megjelenítésére és a letiltott napok vizuális kezelésére - modern szoftvermérnöki gyakorlatot követve – nem saját naptárat készítettem, hanem a könnyűsúlyú Flatpickr [25] JavaScript könyvtárat integráltam, így egy nyílt forráskódú, kész intervallum választóval gyorsítottam a munkát.

#### 3.5.3.3. Aszinkron állapotkezelés és Kliensoldali DOM Manipuláció

A felhasználói élményt jelentősen javítja a kattintások okozta felesleges oldalfrissítések (*synchronous page reloads*) lecsökkentése, amit az aszinkron adatkezeléssel értem el.

A publikus galéria felületén a képek szűrése pillanatok alatt, az oldal újratöltése nélkül megtörténik, natív asztali alkalmazások (*native apps*) teljesítményét szimulálva webes környezetben. A szűrés Vanilla JavaScripttel és HTML5 adat-attribútumokkal (*data-filter*, *data-category*) történik. A CSS tranzícióknak (*opacity + setTimeout*) köszönhetően ráadásul a képek nem csak eltűnnek, hanem finoman elhalványulnak, megakadályozva a vizuális ugrálásokat. Ezzel a rendszerrel az interakciós költség minimálisra csökken, és a megszakítás nélküli (*seamless*) élmény dominál.

## 4. A rendszer tesztelése

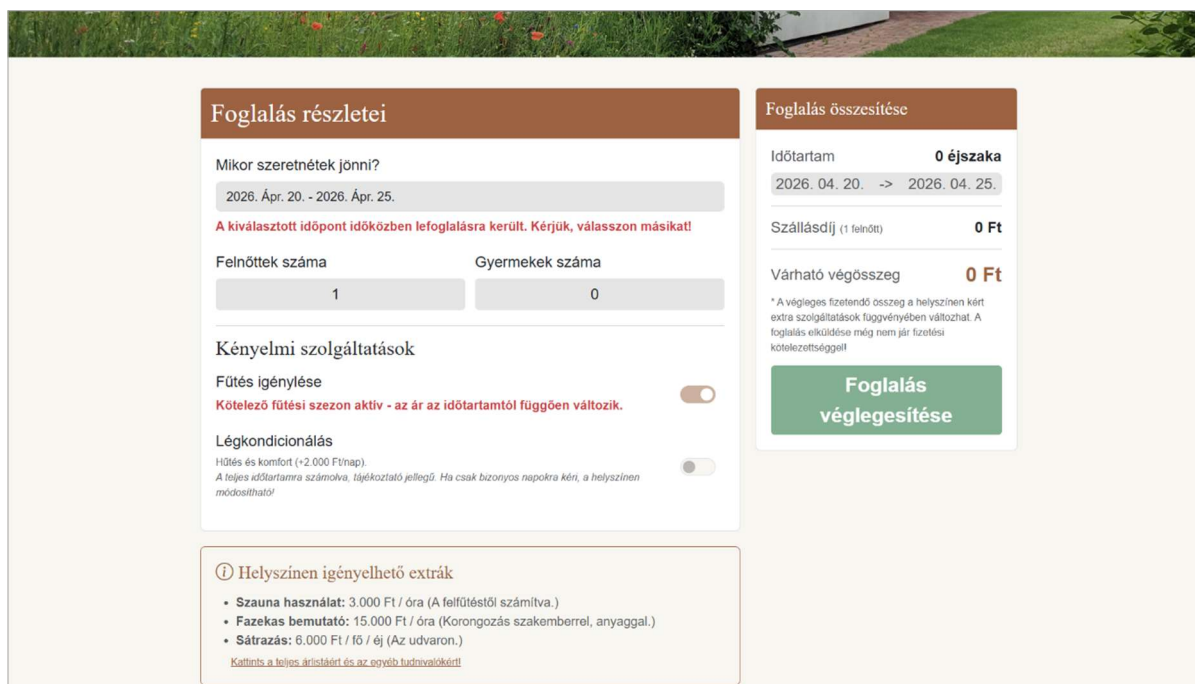
Mivel a rendszer élesítése (*deployment*) még nem történt meg, ezért a Google Lighthouse eszközzel a teljesítményméréseket nem végeztem el, hiszen az eredmény nem tükrözné a valóságot. Ehelyett manuális tesztek hajtottam végre szélsőséges esetekkel és rosszindulatú aktorként viselkedve.

### 4.1. Üzleti logika és funkcionális tesztek

Az üzleti logika tesztelése során kiemelt fókuszot kapott a foglalási szabályok szigorú betartatása és a kivételes esetek megfelelő kezelése.

A rendszerben a minimális foglalás mértékét 1 éjszakában határoztam meg, és ennek van egy 25%-os felára, amit az *Árak & Kapcsolat* oldal harmonika menüjében (*accordion*) olvashat a felhasználó, illetve a kliensoldal dátumválasztója sem fogad el helytelen értéket. Tesztelés során szándékosan a minimálisan megengedett intervallumot választottam ki. A tesztelés sikeressége a foglalás összesítő szegmensében is már megmutatkozott, miszerint az árszámító algoritmus felismerte a határértéket (*boundary value*), és helyesen alkalmazta a felárat a részösszegekre és a végösszegre is.

Kialakulhat olyan scenárió is, amikor a felhasználó szabadnak lát egy időpontot, mert nyitva hagyta órákig a böngészőjét, de mire kiválasztaná a dátumokat, azok már le vannak foglalva. Ezen állapotütközések feloldására szolgál a rendszerbe épített versenyhelyzet- (*race condition*) és konkurenciakezelés (*concurrency handling*). A túlfoglalás (*overbooking*) megelőzésének érdekében a backend aszinkron módon, már az intervallum kiválasztását követően, még mielőtt megjelenne az ár az összesítőben, megvizsgálja az adatbázisban, hogy szabad-e az időpont, és amennyiben nem, hibaüzenetet ír ki a felhasználónak (19. ábra). Ugyanez történik egy másik esetben is: mivel a vendégházat délelőtt 10 órakor kell elhagyni, és délután 2-kor foglalható el, egy adott foglalás első, és egy másik utolsó dátuma megegyezhet, tehát azt szabadnak mutatja a publikus felület. Az egyéjszakás foglalás esetén viszont egyik napot sem mutatja foglaltnak, a fenti üzleti logikai sajátosságból (átfedésből) adódóan. Ha ez a két nap beleesik a foglalásba, akkor szintén hibaüzenetet küld vissza a szerver. Végül abban az esetben, amikor a **Foglalás véglegesítése** gomb megnyomásakor bukik meg a dátum, a szerveroldali validációs réteg (*validation layer*) felismeri a logikai ütközést, és a tranzakciót biztonságosan megszakítva értesíti a felhasználót.



**Foglalás részletei**

Mikor szeretnétek jönni?  
2026. Ápr. 20. - 2026. Ápr. 25.

A kiválasztott időpont időközben lefoglalásra került. Kérjük, válasszon másikat!

Felnőttek száma: 1      Gyermekek száma: 0

**Kényelmi szolgáltatások**

Fűtés igénylése  
Kötelező fűtési szezon aktív - az ár az időtartamtól függően változik. ☒

Légkondicionálás  
Hűtés és komfort (+2.000 Ft/nap).  
A fűtési időtartamra számolva, tájékoztató jellegű. Ha csak bizonyos napokra kéri, a helyszínen módosítható! ☐

**Helyszínen igényelhető extrák**

- Szauna használat: 3.000 Ft / óra (A felfűtéstől számítva.)
- Fazekas bemutatás: 15.000 Ft / óra (Korongozás szakemberrel, anyaggal.)
- Sátározás: 6.000 Ft / fű / éj (Az udvaron.)

Kattints a teljes árlistáért és az egyéb tudnivalókat!

**Foglalás összesítése**

Időtartam: 0 éjszaka  
2026. 04. 20. -> 2026. 04. 25.

Szállásdíj (1 felnőtt): 0 Ft

Várható végösszeg: 0 Ft

\* A végleges fizetendő összeg a helyszínen kért extra szolgáltatások függvényében változhat. A foglalás elküldése még nem jár fizetési kötelezettséggel!

**Foglalás véglegesítése**

19. ábra: Hibaüzenet a dátumválasztás folyamatában, időközben lefoglalt dátum esetén

## 4.2. Biztonsági tesztelés és a Zéró Bizalom

A minőségbiztosítás másik fontos egysége a manipulált kliensoldali kérések és az illetéktelen behatolási kísérletek (*penetration testing*) elleni védelem verifikálása.

A teszt során a böngésző fejlesztői eszközeivel (*DevTools*) szándékosan, DOM manipulációval az alapértelmezetten 1 minimum értékkel rendelkező felnőttek számát 0-ra módosítottam, és 0 értékkel küldtem el az űrlapot. A rendszer megtagadta a feldolgozást, de nem adott részletes hibaüzenetet. Ez egy tudatos biztonsági döntés, amit a kiberbiztonságban információ-elrejtésnek (*information hiding / error obfuscation*) hívnak. A rendszer elkapta a hibát, de csak egyszerűen, mindenféle hibaüzenet nélkül visszairányított a foglalási űrlapra. Ezzel a biztonságos hibaállapot (*fail securely*) elve érvényesül: a rosszindulatú aktor számára nincs felfedett hibaüzenet, amiből egyértelmű lenne, hogy pontosan milyen validációs szabályok futnak a háttérben.

Egy másik teszt során a kötelező fűtési szezon kapcsolójának biztonságát teszteltem. Amikor ez aktív, akkor a felhasználónak nincs választása, mindenképp beleszámít az árba a fűtési felár.

A kísérlet folyamán az aszinkron HTTP kérés elfogásával (*interception*) és a JSON adatcsomag payload-jának módosításával a `wants_heating` értéket `false`-ra állítottam be. Ezzel a teszttel a zéró bizalom architektúra hatékonyságát teszteltem. Bár a kliensoldal megjelenítését sikerült manipulálni, a foglalás összesítője nem mutatta a felárat, és a foglalás is sikeresen végbement, de az adatbázisba mégis a helyes ár került be, és a `wants_heating` is pozitív értéket kapott. Ez azért történt így, mert a Controller által használt árkalkulációs Szolgáltatás az adatok validálása mellett a fűtés és klíma kapcsolók kötelező voltát illetve elérhetőségét is ellenőrzi, és az adatbázisban szereplő adat alapján dolgozik, ami így egyetlen igazságforrásként funkcionál. A felhasználó a kapott automatikus hitelesítő emailből szembesülhet a manipulációs kísérlet megghiúsulásáról, amiben szintén a helyes adatok szerepelnek.

## 5. Összefoglaló

### 5.1. Konklúzió

A fejlesztés megkezdése előtt a célkitűzések között szerepelt egy modern, független, testreszabható foglalási rendszer létrehozása, amely zökkenőmentes élményt nyújt a vendégek számára, miközben az adminisztrációs munkafolyamatok automatizálásával drasztikusan megkönnyíti a tulajdonos dolgát, ezzel valós üzleti értéket (*business value*) teremtve a projektnek. Azon túl, hogy ezt sikeresen teljesítettem, a projekt túlnőtt egy egyszerű weboldalon: egy monolitikus webalkalmazást (*monolithic web application*) hoztam létre, amely stabil MVC architektúrára épül.

A fejlesztés során sokat tanultam a Laravel backend erejéről, az MVC architektúra előnyeiről, és a reszponzivitást előtérbe helyező *Mobile First* szemléletről. Gyakorlatban is tapasztalhattam, hogy mennyire fontos – a zero trust elv alapján, amely garantálja az adatintegritást, – a háttérben ellenőrizni minden eltárolandó adatot. Betekintést nyertem abba, hogy mennyire megkönnyíti a Laravel felelősségi körök szétválasztása elvet követő mappastruktúrája a kódírást, a kódkarbantarthatóságot (*maintainability*), vagy akár más fejlesztők későbbi beavatkozását. A Bootstrap keretrendszer alkalmazása pedig arra ösztönzött, hogy ne csak a fejlesztés során, asztali környezetben mutasson jól a weboldal, ami konzekvens, reszponzív felülettervezést eredményezett.

A fejlesztés közben felmerülő összetett logikai részek, mint például az ár kalkuláció, szintén rávilágítottak a Laravel MVC architektúrájának és a beépített elemeinek előnyeire. Megmutatta, hogy a Vezérlőből a Service rétegbe kiszervezett üzleti logika mennyire megnöveli az átláthatóságot, egyszerűsíti a hibakeresést, továbbá rávilágított a validáció fontosságára, amit akár több helyen is meg kell csinálni, ezzel is garantálva a rendszer biztonságát. Utóbbi vállalja a felelősséget azért, hogy a táblákba ne kerüljenek ellentmondásos adatok, ami alátámasztja azt is, hogy az adatbázis nem csupán tároló, hanem a rendszer egyetlen hiteles forrása.

A tapasztalaton és tanulmányokon alapuló webfejlesztési, adatbázis-optimalizálási, jogosultságkezelési és számos további alapelv biztosították a megfelelő végeredményt, és a rendszer skálázhatóságát a jövőben. A kutatómunka és a szakirodalom feldolgozása során

ezen paradigmákról elsajátított tudás alapján előre kiszámítható volt egy-egy kódrészlet működése, megbízhatósága, és ezek garantálták az ideális végkifejletet.

Az elkészült program kiberbiztonsági és felhasználói élmény szempontjából is megfelel az előzetes céloknak, jelenlegi formájában azonnal bevezethető (*production-ready*), és amellett, hogy növeli a forgalmat, jelentős adminisztrációs terhet vesz le a tulajdonos válláról.

## 5.2. Továbbfejlesztési lehetőségek

A szoftverfejlesztés egy folyamatos, iteratív életciklus, így egyetlen szoftver sincs kész, végleges állapotban, mindig vannak továbbfejlesztési lehetőségek, amikhez a rendszer moduláris felépítése kiváló alapot biztosít.

Egy jövőbeli új funkció lehet a weboldalon az online fizetés lehetősége foglalás leadásakor, ezzel is maximalizálva annak kényelmét. Ez egy külső fizetési szolgáltató (*payment gateway*, pl. Stripe, Barion vagy SimplePay) API szintű integrációja lenne. Miután sikeresen megtörtént a fizetés, a fizetési átjáró *webhookokon* keresztül, a háttérben, aszinkron módon értesítené a szervert a sikeres tranzakcióról. A vendégnek is előnyös, hiszen nem szükséges megnyitni egy külön alkalmazást, beírni a számlaszámot és a többi adatot, ezzel rengeteg időt és erőfeszítést megspórolva, csökkentve az adminisztrációs súrlódást, ami egy pozitív élmény már a nyaralás, kikapcsolódás megkezdése előtt.

Egy következő fejlesztés a külföldi turisták bevonását célozza. A Laravel beépített nyelvi fájljaival a publikus felület és az automatikus e-mailek több nyelven (pl. angol, német, horvát) is elérhetővé válnának. Ez a megoldás azért is előnyös, mert a Laravel lokalizációs rétegének (i18n) köszönhetően a kód duplikálása nélkül is kivitelezhető a nemzetköziesítés (*internationalization*). Ez különösen fontos lehet, hiszen a vendégház a horvát határon helyezkedik el. Ezzel a lépéssel megnövelhető a vendégek száma, elkerülhetők az automatikus, vagy manuális fordításból adódó félreértések, és növelhető a profi weboldal hatása.

Fejlesztési tervek között szerepel továbbá egy online vendégkönyv is. Egy érdeklődőt nagyban befolyásolhatnak a már tapasztalt, korábbi vendégek véleményei. Amennyiben ezeket megjelenítjük a felületen, a foglaló nagyobb bizalommal állhat a vendéglátó egységhez. Erre egy jó megoldás lehet egy, a foglalás végét követően küldött automatikus email, amiben a vendég egy korlátozott ideig elérhető linken keresztül elmondhatja tapasztalatait, élményeit,

meglátásait, ezzel segítve a tulajdonosnak, és a potenciális vendégeknek egyaránt. Ezen a felületen felhasználók által generált tartalom (*User-Generated Content – UGC*) lenne megtalálható, ami egy kritikus támadási felületet (*attack surface*) jelenthetne. Ez azonban szigorú biztonsági felülvizsgálattal kiküszöbölhető: HTML Purifier segítségével szanitizálni kell a bejövő adatokat az XSS támadások elkerülése végett, és egy adminisztrátori moderációs felületet is ki kell alakítani hozzá.

## 6. Irodalomjegyzék

- [1] The PHP Group, „PHP (v8.5.4)”. [Online]. Elérhető: <https://www.php.net>. (Utolsó elérés: 2026. 02. 08.)
- [2] Otwell, Taylor et al., „Laravel Framework (v12.x)”. [Online]. Elérhető: <https://laravel.com>. (Utolsó elérés: 2026. 03. 15.)
- [3] Bootstrap Core Team, „Bootstrap (v5.3)”. [Online]. Elérhető: <https://getbootstrap.com>. (Utolsó elérés: 2026. 03. 12.)
- [4] Oracle Corporation, „MySQL Database Server”. [Online]. Elérhető: <https://www.mysql.com>. (Utolsó elérés: 2026. 03. 01.)
- [5] Chart.js Contributors, „Chart.js – Open source HTML5 Charts (v4.5.1)”. [Online]. Elérhető: <https://www.chartjs.org>. (Utolsó elérés: 2026. 03. 04.)
- [6] Hamano, Junio C. et al., „Git (v2.60.0)”. [Online]. Elérhető: <https://git-scm.com>. (Utolsó elérés: 2026. 02. 03.)
- [7] M. Fowler, „GUI Architectures,” Martin Fowler blik, 2006. [Online]. Elérhető: <https://martinfowler.com/eaDev/uiArchs.html>. (Utolsó elérés: 2026. 02. 12.)
- [8] Figma, Inc., „Figma Design Tool”. [Online]. Elérhető: <https://www.figma.com>. (Utolsó elérés: 2026. 02. 05.)
- [9] A. Hunt és D. Thomas, „The Pragmatic Programmer: Your Journey to Mastery, 20th Anniversary Edition”. Addison-Wesley Professional, 2019. [Online]. Elérhető: <https://pragprog.com/titles/tpp20/the-pragmatic-programmer-20th-anniversary-edition/>. (Utolsó elérés: 2026. 03. 15.)
- [10] E. Evans, „Domain-Driven Design Reference: Definitions and Pattern Summaries,” Domain Language, 2015. [Online]. Elérhető: <https://domainlanguage.com/ddd/reference/>. (Utolsó elérés: 2026. 02. 18.)
- [11] S. McConnell, „Code Complete: A Practical Handbook of Software Construction, 2nd ed”. Microsoft Press, 2004.
- [12] M. Kleppmann, „Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems”. O'Reilly Media, 2017.
- [13] D. B. Johnsson, D. Deogun, és D. Sawano, „Secure by Design”. Manning Publications, 2019.

- [14] M. Fowler, „Infrastructure As Code,” Martin Fowler bliki, 2016. [Online]. Elérhető: <https://martinfowler.com/bliki/InfrastructureAsCode.html>. (Utolsó elérés: 2026. 02. 22.)
- [15] Meno, Matias (enyo) et al., „Dropzone.js – drag'n'drop library (v5.9.3)”. [Online]. Elérhető: <https://www.dropzone.dev>. (Utolsó elérés: 2026. 03. 02.)
- [16] Tiny Technologies, Inc., „TinyMCE – The world's #1 rich text editor (v8.3.2)”. [Online]. Elérhető: <https://www.tiny.cloud>. (Utolsó elérés: 2026. 03. 03.)
- [17] R. Anderson, „Security Engineering: A Guide to Building Dependable Distributed Systems, 3rd ed”. John Wiley & Sons, 2020.
- [18] NIST (National Institute of Standards and Technology), „Zero Trust Architecture,” NIST Special Publication 800-207, 2020. [Online]. Elérhető: <https://csrc.nist.gov/publications/detail/sp/800-207/final>. (Utolsó elérés: 2026. 02. 25.)
- [19] OWASP Foundation, „Cross-Site Request Forgery Prevention Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Elérhető: [https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site\\_Request\\_Forgery\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html). (Utolsó elérés: 2026. 02. 26.)
- [20] ViteJS Team, „Features: Hot Module Replacement,” Vite.js Documentation. [Online]. Elérhető: <https://vitejs.dev/guide/features.html#hot-module-replacement>. (Utolsó elérés: 2026. 03. 01.)
- [21] Google Chrome Developers, „Time to First Byte (TTFB),” web.dev, 2023. [Online]. Elérhető: <https://web.dev/articles/ttfb>. (Utolsó elérés: 2026. 03. 12.)
- [22] Google Chrome Developers, „Cumulative Layout Shift (CLS),” web.dev, 2023. [Online]. Elérhető: <https://web.dev/articles/cls>. (Utolsó elérés: 2026. 03. 12.)
- [23] W3C (World Wide Web Consortium), „Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, 2023. [Online]. Elérhető: <https://www.w3.org/TR/WCAG22/>. (Utolsó elérés: 2026. 03. 14.)
- [24] D. Norman, „The Design of Everyday Things: Revised and Expanded Edition”. Basic Books, 2013.
- [25] Petrosyan, Gregory (chmln) et al., „Flatpickr – lightweight, powerful javascript datetimestructure (v4.6.13)”. [Online]. Elérhető: <https://flatpickr.js.org>. (Utolsó elérés: 2026. 03. 05.)